

Module «Data Science»

Week 04: Part I



mario.gellrich@zhaw.ch

Dr. Mario Gellrich

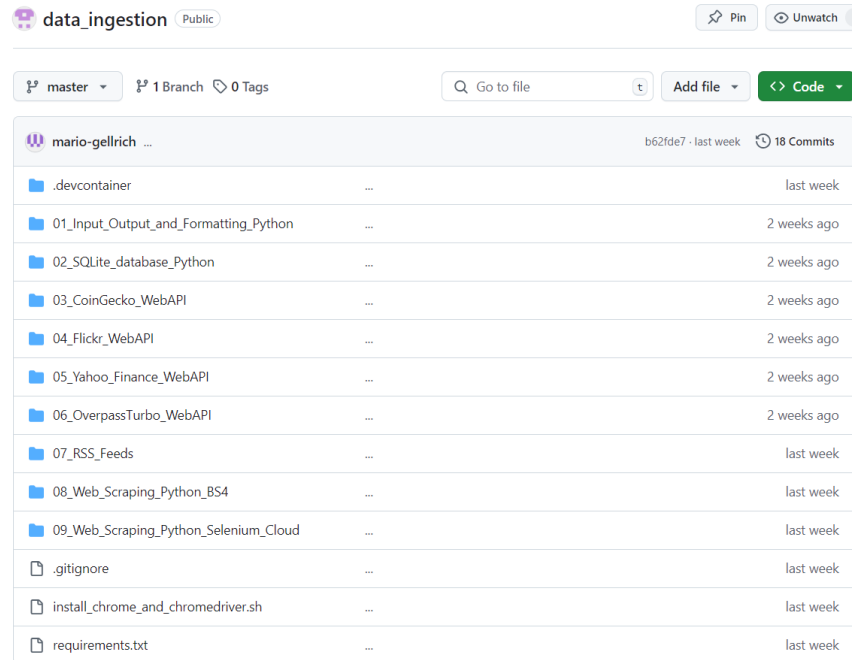
Content of this lecture

Table of contents:

- Data Sources
- Data Types and Structures
- File Formats
- Web Scraping with Python and Selenium
- Collecting Data with Web APIs
- Collecting Data from RSS Feeds

Software used in this Course

GitHub Repository with Course Material for Python exercises



The screenshot shows the GitHub repository page for 'data_ingestion' by user 'mario-gellrich'. The repository is public and has 1 branch and 0 tags. The file list includes:

File/Folder	Last Commit
.devcontainer	last week
01_Input_Output_and_Formatting_Python	2 weeks ago
02_SQLite_database_Python	2 weeks ago
03_CoinGecko_WebAPI	2 weeks ago
04_Flickr_WebAPI	2 weeks ago
05_Yahoo_Finance_WebAPI	2 weeks ago
06_OverpassTurbo_WebAPI	2 weeks ago
07_RSS_Feeds	last week
08_Web_Scraping_Python_BS4	last week
09_Web_Scraping_Python_Selenium_Cloud	last week
.gitignore	last week
install_chrome_and_chromedriver.sh	last week
requirements.txt	last week

[Open the data_ingestion repository on GitHub](#)

Tasks:

- Fork the repository into your own GitHub account
- Create a new Codespaces environment

Data Sources

Introduction to data sources

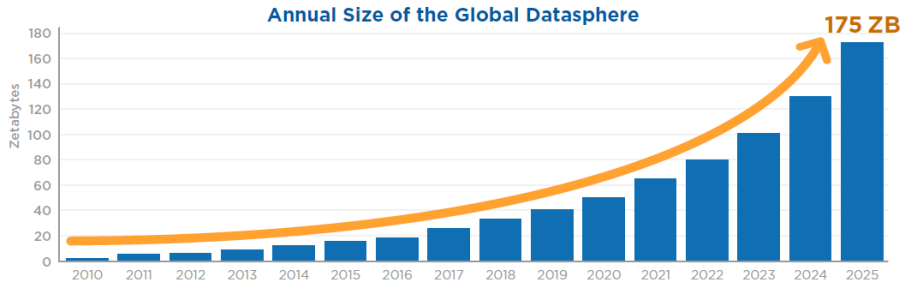
Common data sources

- Primary data sources
- Secondary data sources

Open Data

Introduction to data sources: global data volume

Global data volume (development from 2010 and forecast up to 2025)



1 Zettabyte (ZB) = 1 billion terabytes (TB)

For illustration: 1 TB external hard disk with 12 cm length
 $0.00012 \text{ (km)} * 175'000'000'000 \text{ (TB)} / 40'075 \text{ (km)} = 524$.
Put all the hard disks together ~ 524 times the earth's circumference.

Introduction to data sources: definitions

What is a data source?

“A data source is the location where data that is being used originates from.”

<https://www.talend.com/resources/data-source>

“DataSource is a name given to the connection set up to a database from a server. The name is commonly used when creating a query to the database.”

<https://en.wikipedia.org/wiki/Datasource>

“The term “data source” can also refer to the characteristics and components of the raw statistical data used for compiling statistical aggregates.”

[OECD Glossary of Statistical Terms: data source](#)

“A data source is an object that represents a repository of data, such as an email archive, a fileshare, or a filing cabinet.”

<https://www.ibm.com>

Introduction to data sources: primary and secondary data

According to their origin, data can be divided into primary and secondary data.

Primary data is a type of data that is collected by researchers directly from main sources through interviews, surveys, experiments, etc. Primary data are usually collected from the source where the data originates from. Examples include results of expert interviews or data from IoT devices or camera drones.



Secondary data is the data that has already been collected through primary sources and made readily available for researchers. Common sources of secondary data include censuses, information collected by government departments, and organizational records stored in databases.



Image credit: shutterstock.com

Primary data sources: Interviews

An **interview** is a qualitative research method that relies on asking questions in order to collect data. Interviews can be structured, semi-structured or unstructured. Interviews are a source of primary data.

For those who think interviews don't provide much data, check out a detailed transcript of a 1-hour interview 😊.



Primary data sources: Field measurements

Field measurements are also a source of primary data. These data are often used for quantitative research which usually involves exploratory and statistical data analysis.



Image Credit: shutterstock.com

Primary data sources: Questionnaires

Questionnaires are another source of primary data used for quantitative research. Data from questionnaires are often ordinal data from a Likert-scale allowing individuals to express how much they agree or disagree with a particular statement.

5 Point Likert Scale

Numbering	1	2	3	4	5
	Strongly Agree	Agree	Neutral	Disagree	Strongly Disagree
I like Stock Market	☐	☐	☐	☐	☐
I like Stocks	☐	☐	☐	☐	☐
I Like Money	☐	☐	☐	☐	☐
I Like Return	☐	☐	☐	☐	☐
Total	4	8	12	16	20

Image credit: <https://www.wallstreetmojo.com/likert-scale/>



Image credit: shutterstock.com

Primary data sources: Physical devices (overview)

Physical devices as primary data source are, for example:

- Sensors (for temperature, air pressure, acceleration)
- Cameras
- Computers
- Smartphones

Physical devices collect and store data (e.g. measurements, photos, videos). The data transfer (e.g. to a web server) is usually made via interfaces.

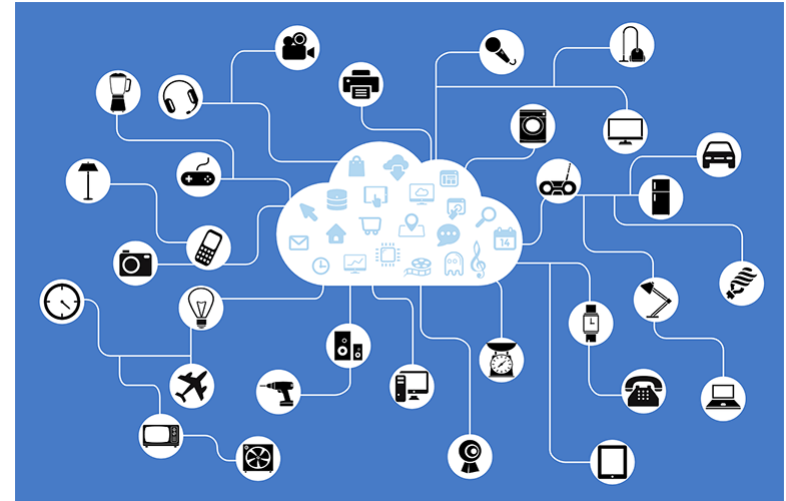


Image credit: <http://safwee.com/en/iot-and-connected-devices>

Primary data sources: Physical devices – drone mapping

Example of primary data collection: high resolution aerial photos from drones for drone mapping



Image Credit: <https://dronephotographyservices.co.uk/mapping/3d-mapping/>

Primary data sources: Physical devices – LIDAR data

Example of primary data collection: LIDAR data

Jucker-Farm Pfäffikon (ZH)



3D-Umgebungsmodelle aus Open Data & Lidar



Abbildung 8: LiDAR Modell, Blickrichtung: Westen

Iselin, A., Simeon, F., Graf, S. (2022). 3D-Umgebungsmodelle aus Open Data & Lidar. Projektarbeit ZHAW (IWI).

Secondary data sources: Web pages

“Your company’s biggest database isn’t your transaction, CRM, ERP or other internal database. Rather it’s the Web itself... Treat the Internet itself as your organization’s largest data source.”

Gartner quotes in <https://www.import.io/post/how-to-get-data-from-a-website>

Data extraction from websites, e.g. via web scraping



Image Credit: <https://dev.to/prsharankumar/web-scraping-using-python-2ip6>

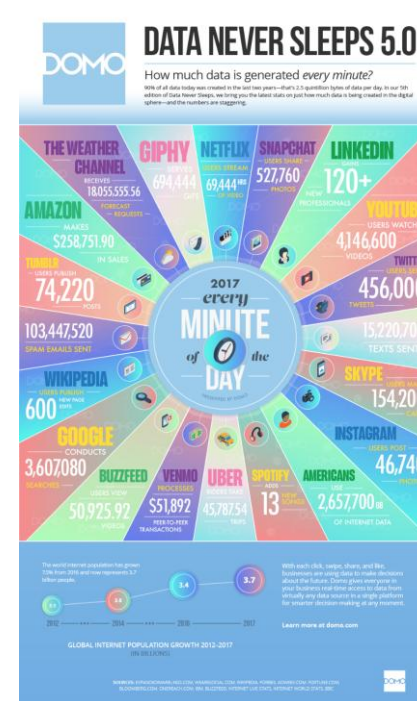


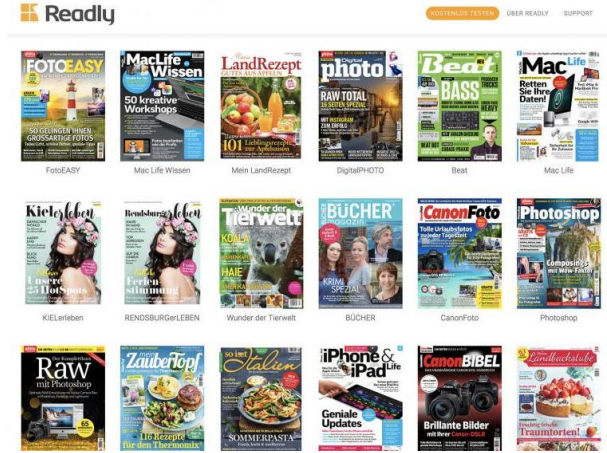
Image Credit: <https://www.domo.com>

Secondary data sources: Publications – publishers and platforms

Already published work like **journal articles**, **newspaper articles** and **books** is another important source of data. Large text corpora are often analyzed using natural language processing (NLP).

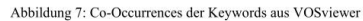


The **academic publishing industry** has a large financial turnover. The market is largely dominated by five large publishing houses: **Elsevier, Wiley-Blackwell, Taylor & Francis, Springer Nature and SAGE**, which control more than 50 % of the market between them.



Newspaper and magazine subscription services like **Readly** provide content from several hundred third-party publishers. As of August 2026, a total of 5000 titles were available on Readly's platform.

Results from a bibliometric analysis based on secondary data (here: scientific publications)



Keyword	Occurrences	Keyword Plus	Occurrences
ELELCTRIC VEHICLES	367	ELELCTRIC VEHICLES	184
ELECTRIC VEHICLE	311	OPTIMIZATION	177
OPTIMIZATION	137	MODEL	160
ELECTRIC VEHICLE CHARGING	123	IMPACT	114
SMART CHARGING	122	MANAGEMENT	114
CHARGING STATION	82	DEMAND	105
CHARGING STATIONS	80	SYSTEM	94
BATTERY POWERED VEHICLES	63	ENERGY	75
ELECTRIC VEHICLE (EV)	55	INFRASTRUCTURE	75
SMART GRID	55	STATIONS	74

Dr. Mario Gellrich, HS2026

Open Data: public and private data

Open Data as an intersection of public and private data

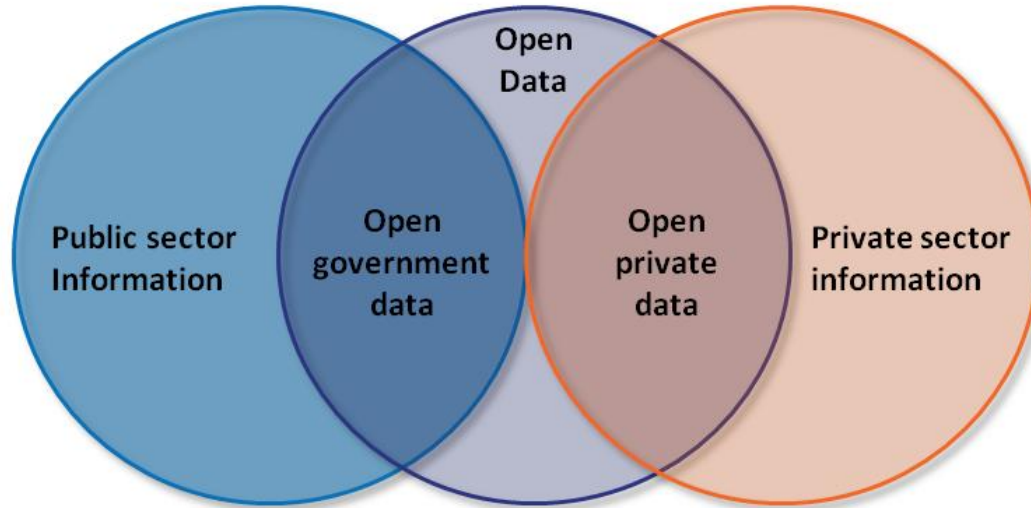


Image Credit: https://data.europa.eu/sites/default/files/analytical_report_n9_economic_benefits_of_open_data.pdf

Open Data: government platforms (EU, CH, DE, USA)

Examples of Open Data Platforms



Europa - Open Data Portal



Offizieller Datenkatalog der Europäischen Union.



Schweiz - geo.admin.ch



Geoport der schweizerischen Bundesverwaltung.



Deutschland - GovData



Offizieller deutscher Datenkatalog.



USA - DATA.gov



Offizieller Datenkatalog der US-Regierung mit Datensätzen aus US-Behörden.

For Details: [Open Data Showroom](#)

Open Data: community platforms (Kaggle, GitHub)

Examples of Open Data Platforms

The screenshot shows the Kaggle homepage with a search bar at the top. The left sidebar contains navigation links: Home, Compete, Data, Notebooks, Discuss, Courses, and More. The main content area displays 'Related Competitions' with two featured competitions: 'COVID19 Global Forecasting (Week 1)' and 'COVID19 Local US-CA Forecasting (Week 1)'. Below these, there are tabs for 'PUBLIC', 'YOUR DATASETS', and 'FAVORITES'. The 'PUBLIC' tab is active, showing a list of datasets. The first dataset is 'COVID-19 Open Research Dataset Challenge (CORD-19)' by Allen Institute For AI, with 3809 upvotes. The second dataset is 'Data Science for COVID-19 (DS4C)' by datarlist, with 813 upvotes.

The screenshot shows the GitHub repository page for 'CSSEGISandData / COVID-19'. The top navigation bar includes links for 'Why GitHub?', 'Enterprise', 'Explore', 'Marketplace', and 'Pricing'. The repository page shows a 'Join GitHub today' banner, followed by a link to 'Novel Coronavirus (COVID-19) Cases, provided by JHU CSSE'. Below this, there are statistics for the repository: 455 commits, 2 branches, 0 packages, 0 releases, and 4 contributors. A table lists the repository's files and their last commit times:

File	Last commit
archived_data	finish test 3 days ago
csse_covid_19_data	adjust daily case 03-22-2020 8 hours ago
who_covid_19_situation_reports	Add files via upload 7 hours ago

Open Data: Swiss platforms

Examples of Swiss open data platforms



Image credit: <https://opendata.swiss/de>

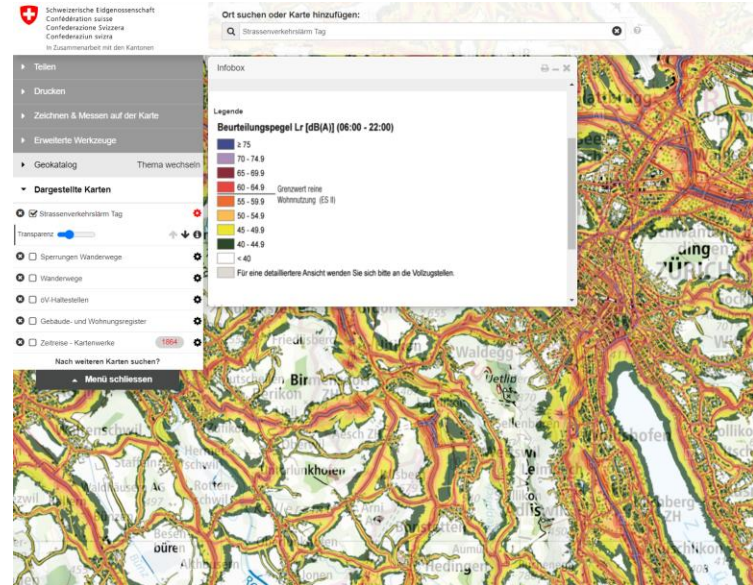


Image credit: <https://map.geo.admin.ch>

Data types & structures

What are data types?

Why data types matter in data analytics

Built-in data types in Python

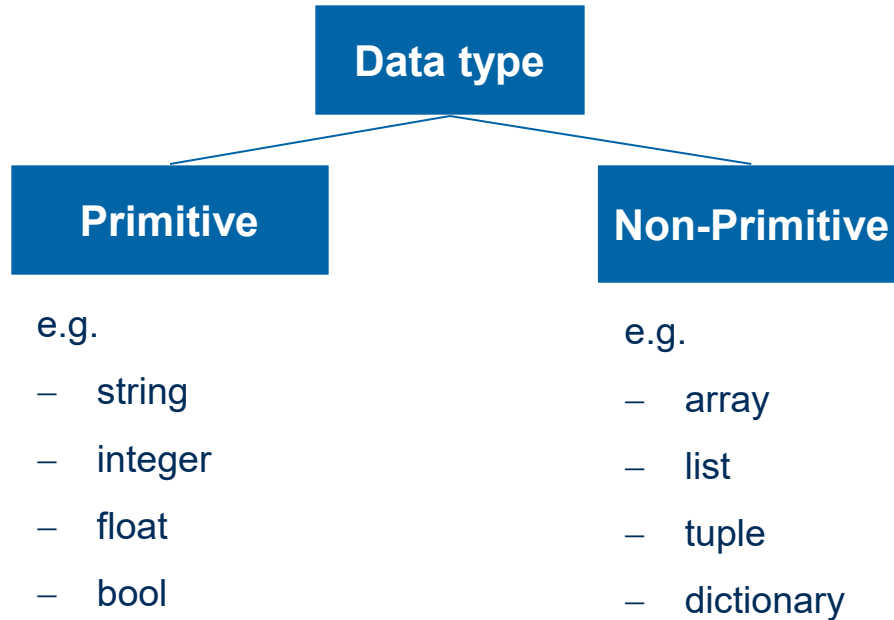
Numeric data types (int, float, complex)

Boolean data type (bool)

Text sequence type (str)

Data structures

What are data types?



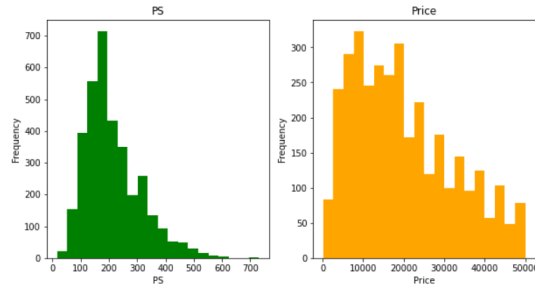
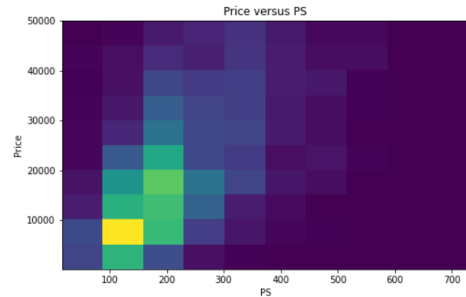
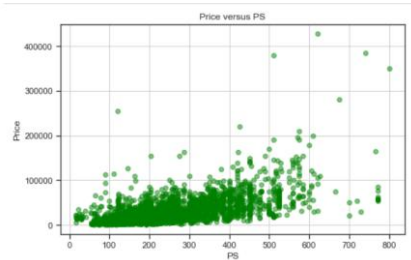
A **data type** is an attribute of data which tells the compiler or interpreter how the programmer intends to use the data. Data types can be divided into **primitive** and **non-primitive** data types. Non-primitive data types are also referred to as composite data types or data structures.

Note: In Python, str is an immutable text sequence type; it is often grouped with the primitive types.

Why data types matter: distribution of numerical data

```
# Show summary statistics of numeric variables  
data.describe()
```

	Offer_Id	Dealer_PLZ	Price_Num	PS_Num	Init_Regist_Month	Init_Regist_Year
count	3.980000e+03	3980.000000	3980.000000	3980.000000	3973.000000	3973.000000
mean	7.242481e+06	5519.984171	27678.180905	236.035427	6.310597	2010.189026
std	4.654468e+05	2848.676121	27280.856305	120.088677	3.385842	9.608932
min	1.395491e+06	1000.000000	160.000000	16.000000	1.000000	1929.000000
25%	7.248346e+06	3014.000000	10900.000000	150.000000	3.000000	2008.000000
50%	7.408630e+06	6003.000000	19900.000000	204.000000	6.000000	2013.000000
75%	7.456746e+06	8304.000000	35900.000000	306.000000	9.000000	2016.000000
max	7.512843e+06	9657.000000	428000.000000	800.000000	12.000000	2020.000000



For example: in practice
you often need to know
how **numerical data**
are distributed ...

Why data types matter: cross tables for categorical data

```
# Show cross tables of categorical variables
pd.crosstab(data.Marke, data.Fuel_Type)
```

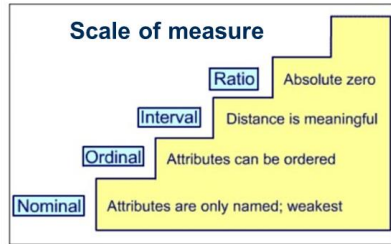
	Fuel_Type	Automat	Automatik-Getriebe	Benzin	Diesel	Elektro	Erdgas (CNG) / Benzin	Flüssiggas (LPG) / Benzin	Hybrid Benzin/Elektro	Hybrid Diesel/Elektro	Mild-Hybrid Benzin/Elektro	Mild-Hybrid Diesel/Elektro	Plug-in-Hybrid Benzin/Elektro	D
Marke														
AC	0	0	0	1	0	0	0	0	0	0	0	0	0	0
ALFA	0	0	0	51	4	0	0	0	0	0	0	0	0	0
ALPINE	0	0	0	2	0	0	0	0	0	0	0	0	0	0
ARMSTRONG	0	0	0	1	0	0	0	0	0	0	0	0	0	0
ASTON	0	0	0	25	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
TVR	0	0	0	1	0	0	0	0	0	0	0	0	0	0
VAUXHALL	0	0	0	1	0	0	0	0	0	0	0	0	0	0
VOLVO	0	0	0	34	42	0	0	0	0	0	0	0	0	3
VW	0	0	0	255	129	0	0	0	0	0	0	0	0	2
WILLYS	0	0	0	1	0	0	0	0	0	0	0	0	0	0

75 rows × 17 columns

For **categorical data**, you may want to count the records per category using cross-tables ...

Why data types matter: choice of statistical method

The choice of the correct statistical method is made on the basis of data types



Examples:

10 kg

36 degree Celsius

bad, neutral, good

cat, dog, bird

Numerical data

Ordinal data

Categorical data

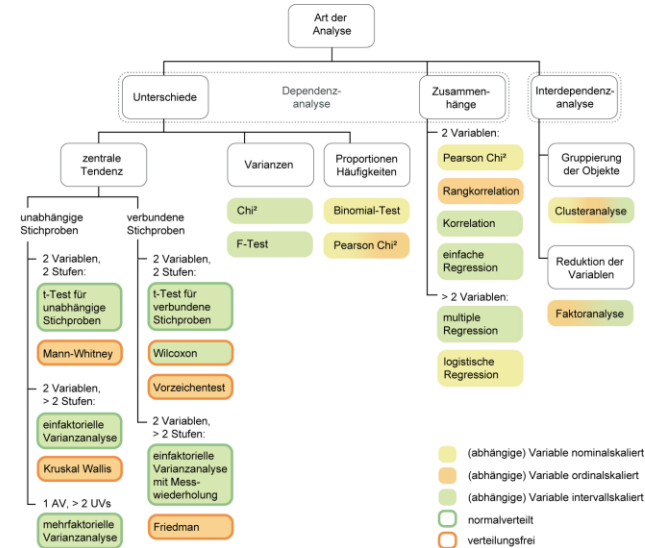


Image Credit:
<https://micheledamato.altervista.org/statistics-research-n-1>

Image credit: <https://www.methodenberatung.uzh.ch>

Why data types matter: ordinal data

The choice of the correct statistical method is made on the basis of data types

Rate Your Understanding

0	1	2	3	4
				
I am so lost.	I don't really get it.	I'm starting to get it.	I got this.	I could teach it.

distance != distance

For example, sometimes, values which look like numerical values are **ordinal values** in reality. Ordinal values require specific statistical methods (e.g. rank correlation such as Spearman, ordinal regression). If not considered, results of data analysis are suspect.

Ordinal variables are just categorical variables where there is an order between levels, but not a meaningful distance between levels.

Why data types matter: choice of machine learning model

The choice of a suitable machine learning model depends on the data types

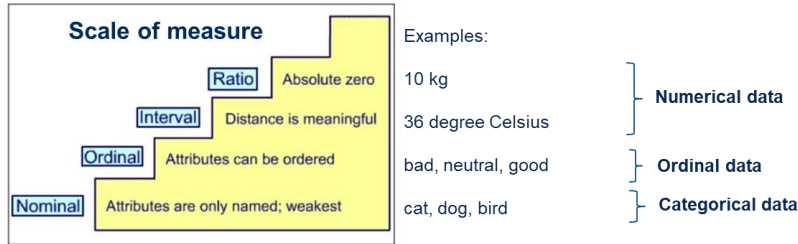


Image Credit:
<https://micheledamato.altervista.org/statistics-research-n-1>

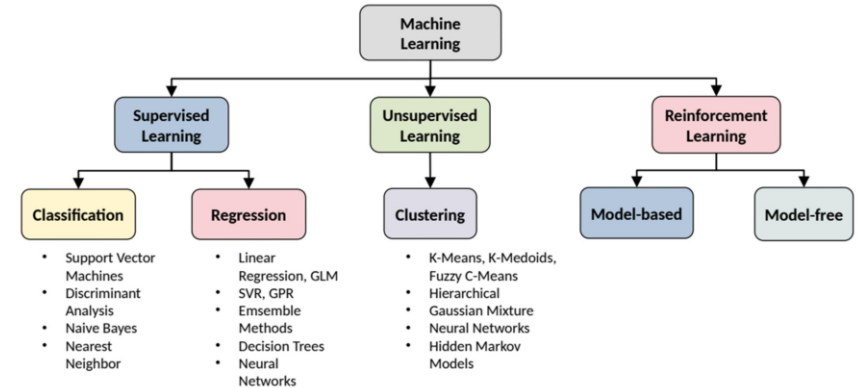


Image credit: <https://www.mdpi.com/2071-1050/13/9/5248>

Built-in data types in Python

Built-in data types (primitive data types) in Python

Numeric types: int, float, complex

Boolean type: bool

Text sequence type: str

Data types in Python: int, float, complex

```
# int (or integer) is a whole number, positive  
# or negative, without decimals and of any length  
x = 200  
y = -20**2  
print(type(x), type(y))  
<class 'int'> <class 'int'>
```

int

```
# Float (or floating-point number) is a number,  
# positive or negative, containing decimals  
x = 3.14  
y = -2.71  
print(type(x), type(y))  
<class 'float'> <class 'float'>
```

float

```
# Complex numbers are written with a "j" as  
# the imaginary part  
x = 3+5j  
y = 5j  
print(type(x), type(y))  
<class 'complex'> <class 'complex'>
```

complex

Data types in Python: bool

Boolean data types represent one of two values: True or False

```
# Comparing two values
print(10 > 9)
True
print(type(10 > 9))
<class 'bool'>

print('cat' == 'dog')
False
print(len('cat') < len('bird'))
True

# Check data type using isinstance()
isinstance(4, int)
True
```

Boolean (bool): When two values are compared, the expression is evaluated, and Python returns the Boolean answer which is True or False.

Strings in Python (str)

In Python, a string (str) is either a single character or a sequence of characters.

```
# Single character string
x = "a"
print(type(x))
<class 'str'>

# Sequence of characters string
car = "JEEP Renegade 2.0 MJ Trailhawk AWD"
print(type(car))
<class 'str'>

# Multiline string
car_desc = """Hubraum: 1'956 cm3 Zylinder: 4 Anhängelast
gebremst: 1'500 kg Aussenfarbe: anthrazit mét.
Innenfarbe: anthrazit Türen: 5 Sitze: 5 Leergewicht:
1'840 kg Typenschein: 1JB347 Fahrzeugart: Occasion"""
print(type(car_desc))
<class 'str'>
```

Strings in Python are surrounded by either single quotation marks, or double quotation marks. 'hello' is the same as "hello". Multiline strings are surrounded by triple quotation marks """multiline string comes here""".

Data structures

A **data structure** is a data organization, management, and storage format

Dimension	Homogeneous data types	Heterogeneous data types
1	vector	list
2	matrix*	data frame
any	array*	

In Python, built-in data structures (composite data types) are:

list = collection, ordered, changeable, indexed
dictionary = collection, ordered (from Python 3.7), changeable, key:value pairs
tuple = collection, ordered, unchangeable, indexed
set = collection, unordered, changeable, elements immutable, unindexed

*Note that in Python it is possible to construct array- and matrix-like objects with more than one data type (e.g. by using nested lists). This is not recommended because mathematical functions applied to these objects are inefficient.

Vector

1	Eier
1	Salz
2	Butter
2	Schmalz
2	

Data frame

ID	Name	Note1
1	Anna	1
2	Anna	1
3	Berta	2
4	Carla	2
...	...	...

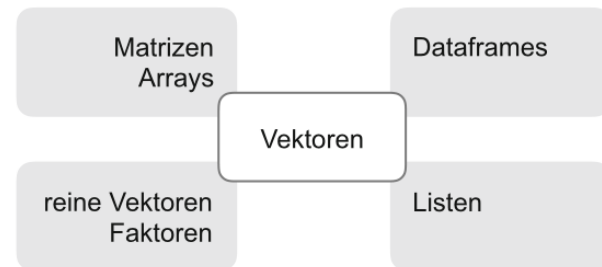


Image credit: Sauer (2019): Moderne Datenanalyse mit R

File formats

Common file formats used for data analytics

- CSV
- TXT
- JSON
- XML
- HTML
- ZIP
- XLSX
- PDF
- Image files
- Parquet

Common file formats

A **file format** defines the syntax and semantics of data within a file.

Common file formats

- CSV
- TXT
- JSON
- XML
- HTML
- ZIP
- XLSX
- PDF
- Image files
- Parquet

CSV (Comma-Separated Values)

A **CSV file (CSV = Comma-Separated Values)** is a text file with separators that usually uses a comma (hence the name!) to separate values.

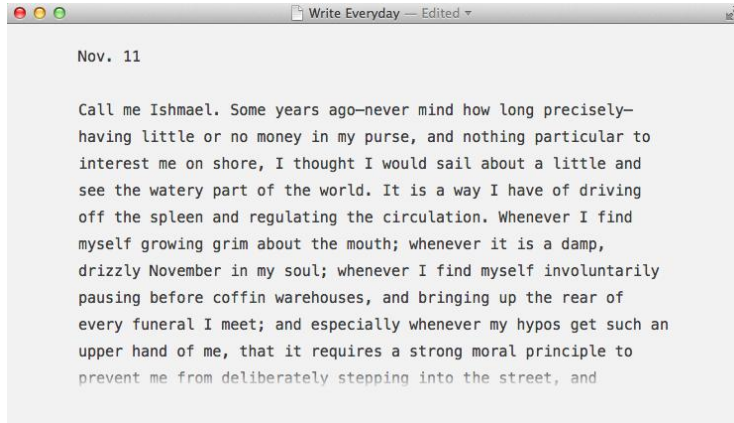
- Each line of a CSV file is one record.
- Each record consists of one or more fields separated by a comma.
- A CSV file typically stores tabular data such as numbers and text in plain text.
- Instead of the comma, a semicolon is also often used as a delimiter.

```
Einkauf; Menge  
Rüebli;10  
Äpfel;5  
Tomaten;8  
Nüsslisalat;1  
Brötchen; 12  
Crème brûlée; 4
```

Example CSV file with header and semicolon as separator

TXT (plain text)

In **plain text file format**, everything is written in plain text. Normally, the text is in unstructured form and there is no metadata associated with it. The TXT file format can be read by most programs.



Example of Plain Text File

Source: <https://zapier.com/blog/plain-text-files-for-productivity/>

JSON (JavaScript Object Notation)

The **JavaScript Object Notation (JSON)** is a compact data format in an easy to read text form and serves the purpose of data exchange between (web) applications.

JSON Syntax Rules

- The data is available in name/value pairs.
- The data are separated by commas.
- Curly brackets contain objects.
- Square brackets contain arrays.

JSON Example

```
{ "employees": [  
  { "firstName": "John", "lastName": "Doe" },  
  { "firstName": "Anna", "lastName": "Smith" },  
  { "firstName": "Peter", "lastName": "Jones" }  
]}
```

XML (Extensible Markup Language)

The **Extensible Markup Language (XML)** is a markup language for representing hierarchically structured data in a text file format that can be read by humans and machines.

XML Syntax Rules

- The XML document has exactly one root element.
- All elements with content have a start and an end tag.
- Elements without content can be marked with an empty tag.
- The start and end tags are properly nested pairs.
- An element may not have multiple attributes with the same name.
- Attribute values must be enclosed in quotation marks.
- The start and end tags are case sensitive.

XML example

```
<bookstore>
  <book category="children">
    <title>Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
  <book category="web">
    <title>Learning XML</title>
    <author>Erik T. Ray</author>
    <year>2003</year>
    <price>39.95</price>
  </book>
</bookstore>
```

HTML (HyperText Markup Language)

HTML (HyperText Markup Language) is the standard markup language for creating web pages and describes the structure of a web page. HTML consists of a series of elements that tell the browser how to display the content. The HTML elements identify the parts of the content of a Web page. HTML is used to structure a text semantically, but not to format it (the browser uses CSS to format the text).

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>This is a Heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

HTML example with head, body, title, h1 and p (paragraph) tags

ZIP (Archive File Format)

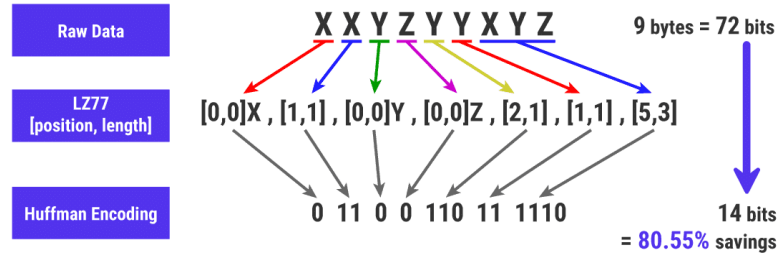


Image Credit: <https://filerev.com/blog/compress-zip-google-drive/>

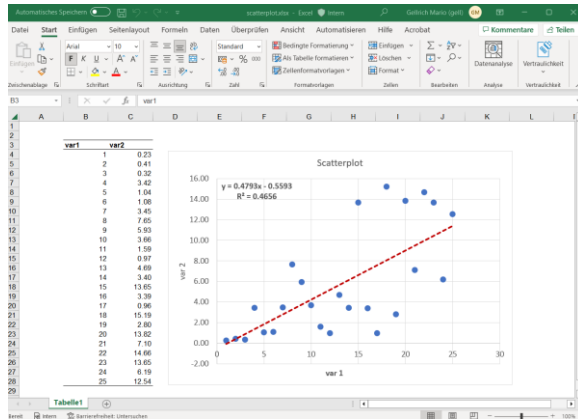
The **ZIP** file format is a format for lossless compressed files, which on the one hand reduces the space required for archiving and on the other hand functions as a container file in which several related files or even entire directory trees can be combined.

XLSX (Microsoft Excel Open XML File Format)

XLSX (Microsoft Excel Open XML file format) is an XML-based file format of Microsoft Excel. The XLSX format was introduced with Microsoft Office 2007. In XLSX ...

- the data are organized in cells, rows and columns of a sheet
- a single workbook can contain several Excel sheets

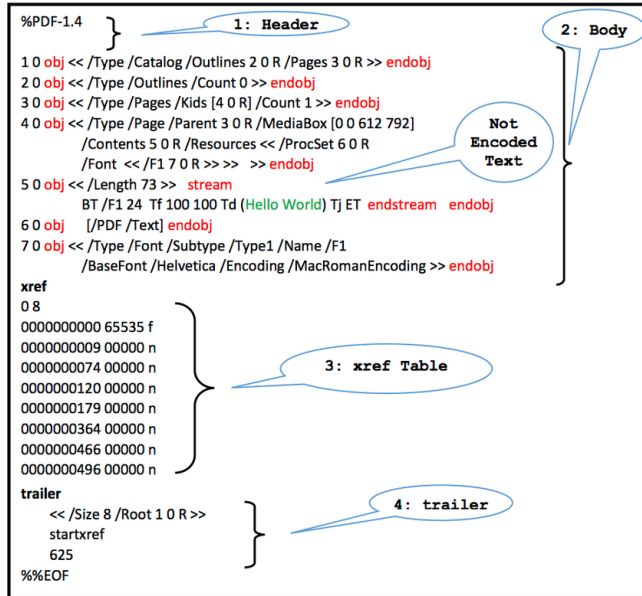
 scatterplot.xlsx



- _rels
- charts
- drawings
- printerSettings
- theme
- worksheets
- sharedStrings.xml
- styles.xml
- workbook.xml

PDF (Portable Document Format)

Example of a PDF file structure



PDF (Portable Document Format) is a file format developed by Adobe in the 1990s to display documents, including text formatting, images, video, audio. Based on the **PostScript** language, each PDF file contains a complete description of a document, including the text, fonts, vector graphics, raster images, and other information required to display the document.

Image Credit: <https://www.researchgate.net/>

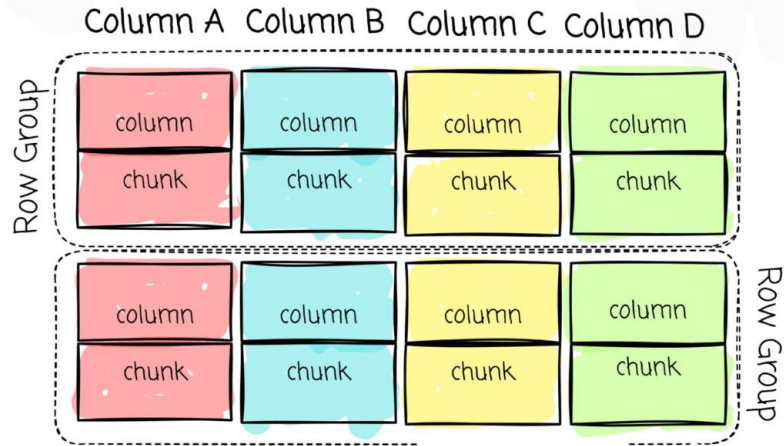
Image files

- Image files consist of pixels and associated metadata.
- Image files can be 2-dimensional (grayscale), 3-dimensional (RGB) or 4-dimensional (RGBA, with an alpha channel for transparency).
- Each image consists of one or more individual frames of pixels.
- Each frame consists of one or more 2-dimensional matrices (channels) of pixel values.
- Pixel values represent the intensity per channel (e.g. 0–255 for 8-bit images).
- Metadata associated with an image can include an image type (e.g. PNG) or pixel dimensions.
- Formats like JPG, GIF, PNG, ... use different compression and encoding methods.



	JPG	GIF	PNG
VECTOR			
RASTER	✓	✓	✓
TRANSPARENCY			✓
ANIMATION		✓	✓
LOSSY	✓		

Parquet: columnar file format



Apache **Parquet** is an open, **columnar** file format for analytical data. Unlike row-oriented formats like CSV, values are stored column by column, compressed, and the schema (column names + data types) is embedded in the file.

Image credit: <https://luminousmen.substack.com/p/why-parquet-is-the-go-to-format-for>

Parquet: comparison with CSV

Example: 1 million apartment records, 6 columns (id, date, city, rooms, area_m2, price_chf), written and read with pandas.

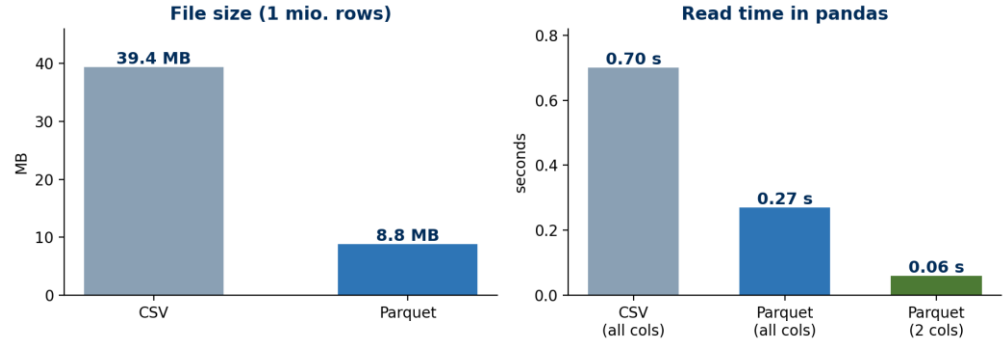
```
import pandas as pd

df = pd.read_csv('apartments.csv')

# Write Parquet
df.to_parquet('apartments.parquet')

# Read back all columns
df = pd.read_parquet('apartments.parquet')

# Read only 2 of 6 columns
df = pd.read_parquet(
    'apartments.parquet',
    columns=['city', 'price_chf'])
```

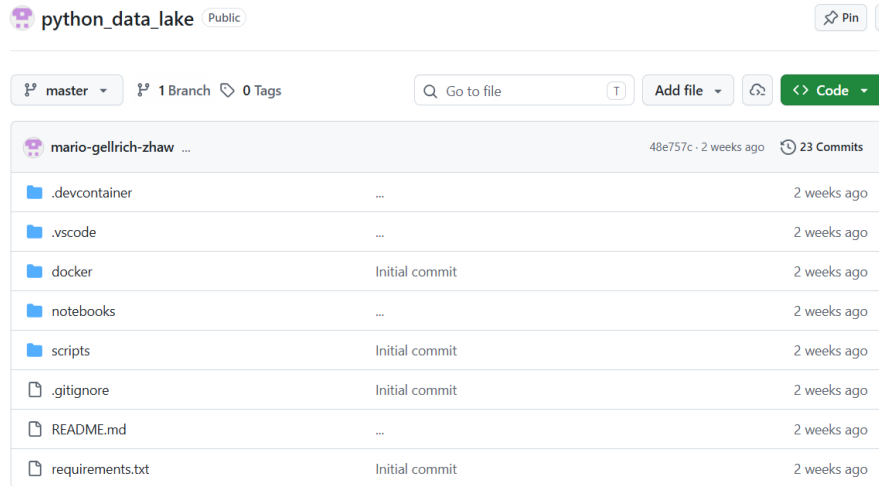


4.5× smaller on disk, 2.6× faster to read — and reading only the columns you need is ~12× faster, because Parquet skips the rest.

Rule of thumb: CSV to exchange data with humans and tools — Parquet to store and analyze large tabular data.

Parquet: GitHub repository for the exercises

GitHub Repository with Course Material for Python exercises



[Open the python_data_lake repository on GitHub](#)

Tasks:

- Fork the repository into your own GitHub account
- Create a new Codespaces environment

Web Scraping with Python and Selenium

Web Mining

Web Browser Functionality

Extracting Data from HTML using XPath Expressions

Selenium WebDriver Architecture

Navigation with Selenium

Locating Elements with Selenium

Web Scraping with Python and Selenium: Examples

Web Mining: Web Crawling

Web Crawling (Web Crawler = Robots = Spider = Bots):

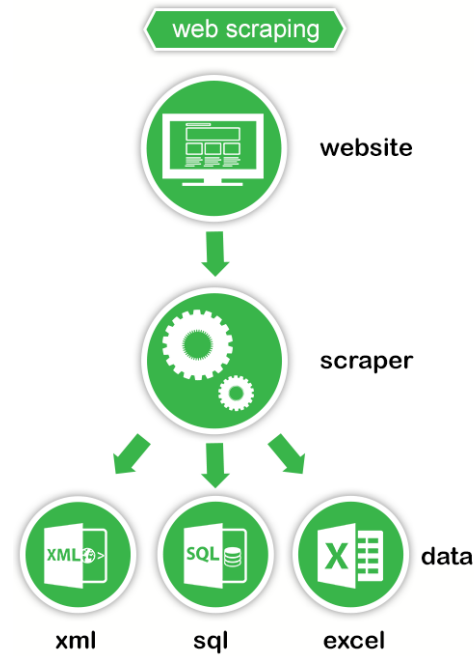
- Process of iterative finding and retrieving of web links
- Web pages and their contents are indexed
- Basis for the good functioning of search engines (Google, Bing, etc.)



Web Mining: Web Scraping

Web Scraping (also Screen Scraping):

- Process of processing a web document and extracting information from it
- Focus on the content of web documents (e.g. product information in online shops, real estate information)



Web Mining: Tools

Without Programming Skills



ScrapeSimple



With Programming Skills

For  python™ Programmer

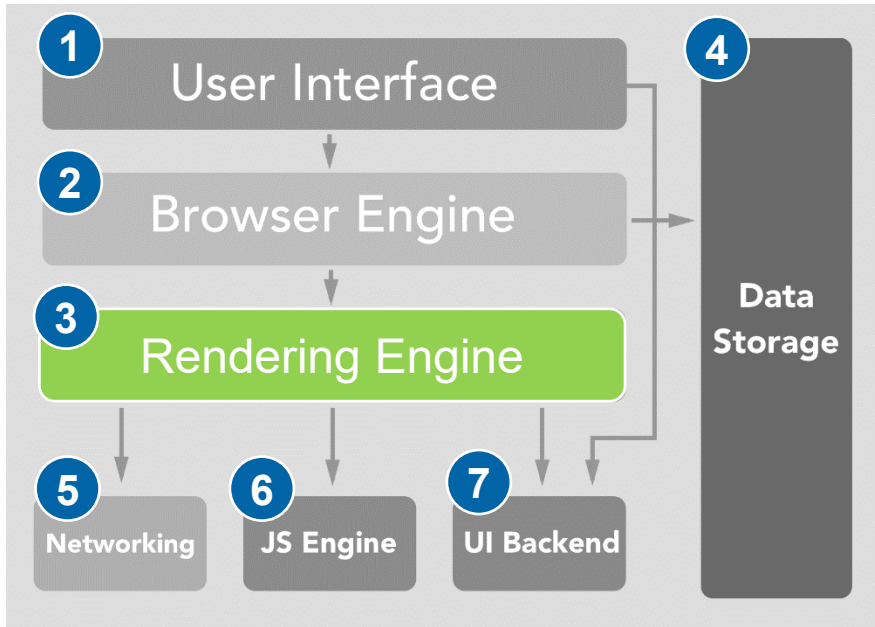


For  R Programmer

Rcrawler
rvest

Web Browser Functionality: components

Web Browser Components



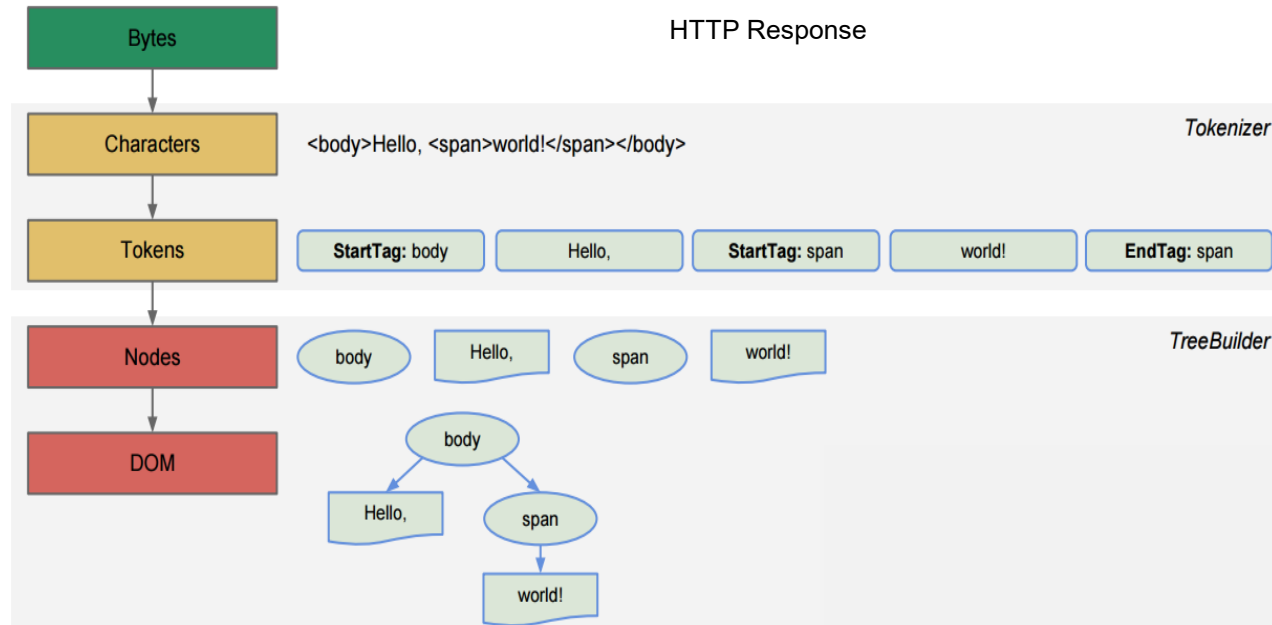
- 1 Input/output (address bar, buttons, ...)
- 2 "Bridge" between UI and rendering engine
- 3 Creation of structure, layout, painting
- 4 Data storage (e.g. cookies)
- 5 Communication with the network
- 6 JavaScript interpreter (e.g. V8 in Chrome)
- 7 Drawing basic widgets in the browser



Source: <https://medium.com/@monica1109/how-does-web-browsers-work-c95ad628a509>

Web Browser Functionality: HTML parser

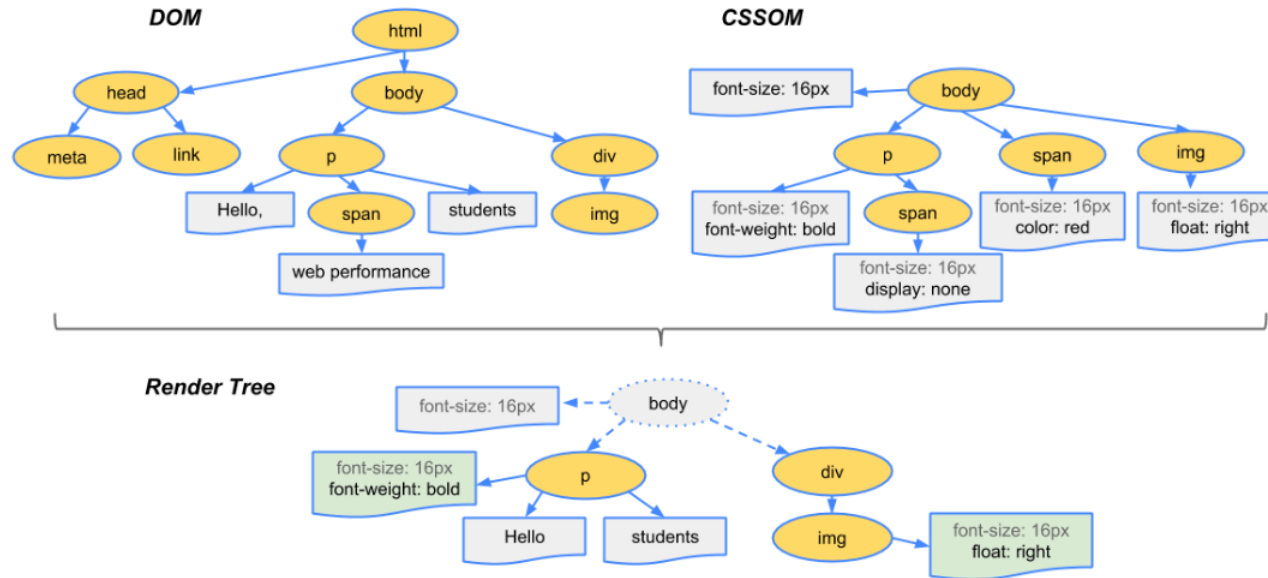
How an HTML parser works



Source: <https://stackoverflow.com/questions/3150293/how-does-a-parser-for-example-html-work>

Web Browser Functionality: rendering process

Rendering process: creating the DOM, CSSOM and the render tree



DOM = Document Object Model

CSSOM = CSS Object Model

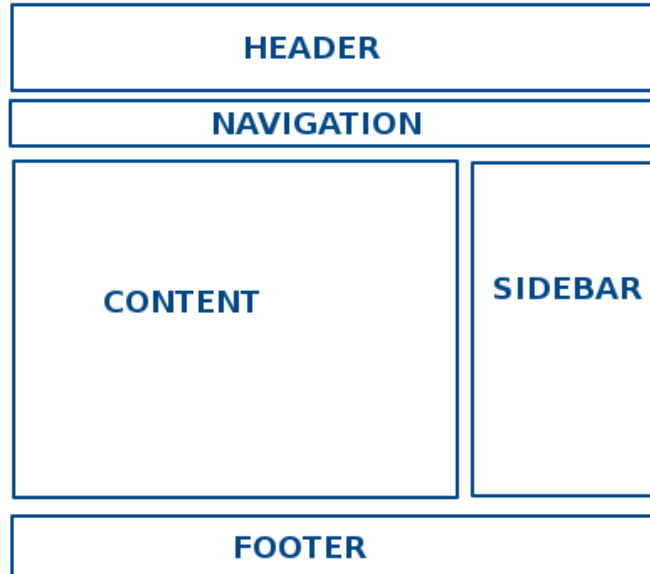
Rendering

Source: <https://developers.google.com/web/fundamentals/performance/critical-rendering-path/render-tree-construction>

Web Browser Functionality: layout stage

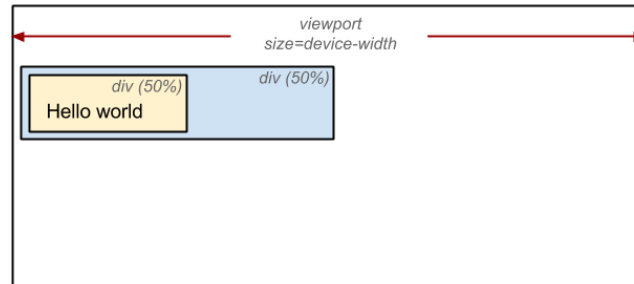
Layout stage (reflow)

Element arrangement



Example: Content with two nested <div> elements

```
<!DOCTYPE html>
<html>
  <head>
    <meta name="viewport" content="width=device-width,initial-scale=1">
    <title>Critical Path: Hello world!</title>
  </head>
  <body>
    <div style="width: 50%">
      <div style="width: 50%">Hello world!</div>
    </div>
  </body>
</html>
```



Extracting Data from HTML using XPath: basics

Data extraction using XPath expressions (reference to nodes in the HTML-tree)

html/body/div/h3

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Book Haven</title>
</head>
<body>
  <h1>Welcome to Book Haven</h1>
  <h2>Featured Books</h2>
  <div>
    <h3>To Kill a Mockingbird</h3>
    <p>by Harper Lee</p>
  </div>
  <div>
    <h3>1984</h3>
    <p>by George Orwell</p>
  </div>
  <div>
    <h3>The Great Gatsby</h3>
    <p>by F. Scott Fitzgerald</p>
  </div>
</body>
</html>
```

html/body/div[1]/h3

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Book Haven</title>
</head>
<body>
  <h1>Welcome to Book Haven</h1>
  <h2>Featured Books</h2>
  <div>
    <h3>To Kill a Mockingbird</h3>
    <p>by Harper Lee</p>
  </div>
  <div>
    <h3>1984</h3>
    <p>by George Orwell</p>
  </div>
  <div>
    <h3>The Great Gatsby</h3>
    <p>by F. Scott Fitzgerald</p>
  </div>
</body>
</html>
```

html/head/title

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Book Haven</title>
</head>
<body>
  <h1>Welcome to Book Haven</h1>
  <h2>Featured Books</h2>
  <div>
    <h3>To Kill a Mockingbird</h3>
    <p>by Harper Lee</p>
  </div>
  <div>
    <h3>1984</h3>
    <p>by George Orwell</p>
  </div>
  <div>
    <h3>The Great Gatsby</h3>
    <p>by F. Scott Fitzgerald</p>
  </div>
</body>
</html>
```

XPath Tester: [XPather](#)

Extracting Data using XPath: autoscout24.ch website

Example Web Scraper for autoscout24.ch data

The screenshot displays the autoscout24.ch website interface. On the left, there are filters for 'Version', 'Marke, Modell hinzufügen', 'Jahr von | bis', 'Preis von | bis', and 'Treibstoff'. The main content area shows two car listings:

- VW ID.3 Pro Performance 58 kWh Tech**
CHF 47'490.-
1'000 km / 09.2020
204 PS
Automatisiertes Schaltgetriebe
Elektro
Der Neue ID.3 jetzt zum Probefahren bereit. Lassen Sie sich Elektrifizieren und steigen Sie in di...
[Online-Beratung](#) [Heimlieferung](#) [+2](#)
[Versicherungsprämie berechnen](#)
- VW ID.3 1ST Max Pro Performance**
CHF 52'800.-
1 km / Neu
204 PS
Automatisiertes Schaltgetriebe
Elektro

Below the listings, the HTML code is visible in a web scraper tool. The code includes various attributes and classes, such as:

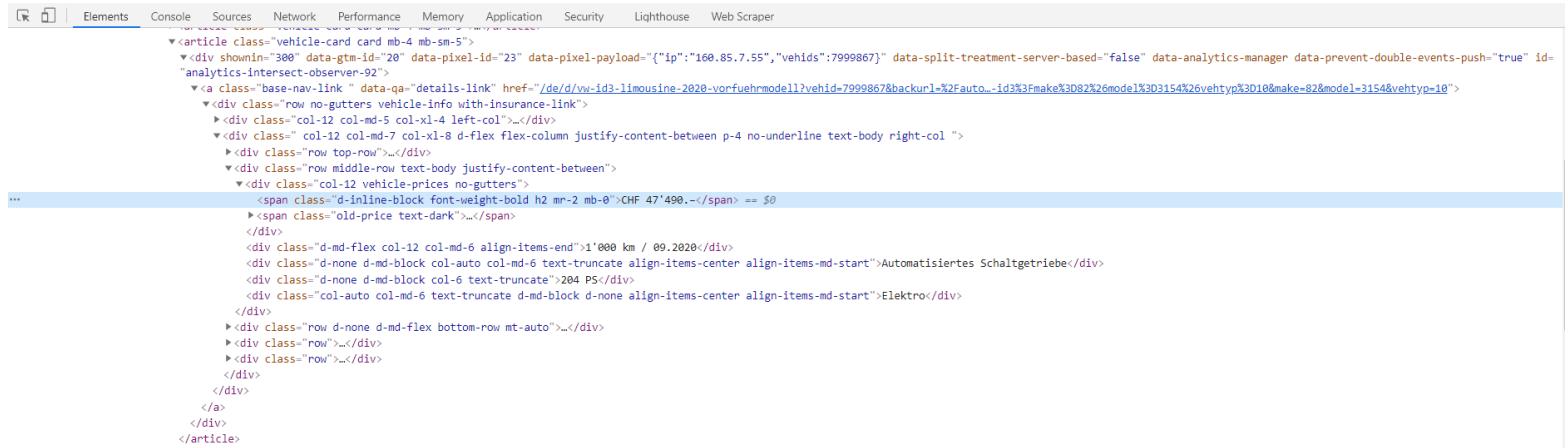
```
<article class="vehicle-card card mb-4 mb-sm-5">  
  <div showin="360" data-gid="28" data-pixi-id="23" data-pixi-payload="{"ip":"100.85.7.55","vid":"7999867"}" data-split-treatment-server-based="false" data-analytics-manager="data-prevent-double-events-push=true" id="analytics-interact-observer-92">  
    <a class="base-nav-link" data-ga="details-link" href="/de/de/nu-1d3-1lineuine-2020-vorfuhrmodell?vehid=7999867&backurl=32&auto_id=37&make=306&2model=3303154&2vehid=3303154&2make=3154&2vehid=3154">  
      <div class="row no-gutters vehicle-info with-insurance-link">  
        <div class="col-12 col-md-5 col-xl-4 left-col">  
          <div class="col-12 col-md-7 col-xl-8 d-flex flex-column justify-content-between p-4 no-underline text-body right-col">  
            <div class="row top-row">  
              <div class="row middle-row text-body justify-content-between">  
                <div class="col-12 vehicle-prices no-gutters">  
                  <span class="d-inline-block font-weight-bold h2 w-2 mb-8">CHF 47'490.-</span>  
                  <span class="old-price text-dark">  
                    <div class="d-md-flex col-12 col-md-6 align-items-end">1'000 km / 09.2020</div>  
                    <div class="d-none d-md-block col-auto col-md-6 text-truncate align-items-center align-items-md-start">Automatisiertes Schaltgetriebe</div>  
                    <div class="d-none d-md-block col-6 text-truncate">204 PS</div>  
                    <div class="col-auto col-md-6 text-truncate d-md-block d-none align-items-center align-items-md-start">Elektro</div>  
                  </div>  
                  <div class="row d-none d-md-flex bottom-row mt-auto">  
                    <div class="row">  
                      <div class="row">  
                        <div>  
                          </div>  
                        </div>  
                      </div>  
                    </div>  
                  </div>  
                </div>  
              </div>  
            </div>  
          </div>  
        </div>  
      </a>  
    </div>  
  </article>
```

Content of the website

HTML – code of the website

Extracting Data using XPath: XPath of the car price

Example Web Scraper for autoscout24.ch data



```
<article class="vehicle-card card mb-4 mb-sm-5">
  <div shownIn="300" data-gtm-id="20" data-pixel-id="23" data-pixel-payload="{"ip":"160.85.7.55","vehids":"7999867"}" data-split-treatment-server-based="false" data-analytics-manager data-prevent-double-events-push="true" id="analytics-intersect-observer-92">
    <a class="base-nav-link" data-qa="details-link" href="/de/d/vw-id3-11mouseing-2020-vorfuhrmodell?vehid=7999867&backurl=%2Fauto.-id%3Fmake%3D82%26model%3D3154%26vehtyp%3D18&make=82&model=3154&vehtyp=18">
      <div class="row no-gutters vehicle-info with-insurance-link">
        <div class="col-12 col-md-5 col-xl-4 left-col"></div>
        <div class="col-12 col-md-7 col-xl-8 d-flex flex-column justify-content-between p-4 no-underline text-body right-col">
          <div class="row top-row"></div>
          <div class="row middle-row text-body justify-content-between">
            <div class="col-12 vehicle-prices no-gutters">
              <span class="d-inline-block font-weight-bold h2 mr-2 mb-0">CHF 47'490,-</span> == $0
              <span class="old-price text-dark"></span>
            </div>
            <div class="d-md-flex col-12 col-md-6 align-items-end">1'000 km / 09.2020</div>
            <div class="d-none d-md-block col-auto col-md-6 text-truncate align-items-center align-items-md-start">Automatisiertes Schaltgetriebe</div>
            <div class="d-none d-md-block col-6 text-truncate">204 PS</div>
            <div class="col-auto col-md-6 text-truncate d-md-block d-none align-items-center align-items-md-start">Elektro</div>
          </div>
          <div class="row d-none d-md-flex bottom-row mt-auto"></div>
        </div>
      </div>
    </a>
  </div>
</article>
```

XPath of the car price (CHF 47'490): `//*[@id="analytics-intersect-observer-92"]/a/div/div[2]/div[2]/div[1]/span[1]`

Selenium WebDriver Architecture: overview

Selenium WebDriver Architecture

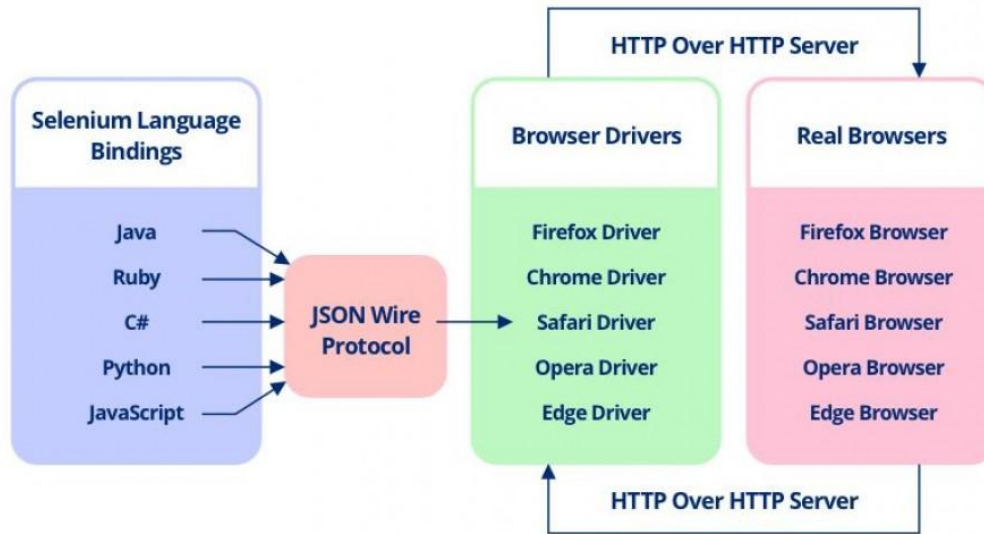


Image Credit: <https://hackr.io/blog/what-is-selenium-webdriver>

Selenium requires a **WebDriver** to interface with **real browsers**. Selenium WebDriver is a collection of **open-source APIs** which are used to automate the testing of web applications. Selenium WebDriver API interacts with real browsers using **browser drivers**.

Note: Since Selenium 4, the W3C WebDriver protocol has replaced the JSON Wire Protocol shown in the diagram.

Selenium WebDriver Architecture: driver and browser versions

Each **browser driver** must match the version of the **real browser**.

Since Selenium 4.6, the built-in Selenium Manager downloads the matching driver automatically.

Example: ChromeDriver and Chrome browser

Stable

Version: 140.0.7339.82 (r1496484)

Binary	Platform	URL
chrome	linux64	https://storage.googleapis.com/chrome-for-testing-public/140.0.7339.82/linux64/chrome-linux64.zip
chrome	mac-arm64	https://storage.googleapis.com/chrome-for-testing-public/140.0.7339.82/mac-arm64/chrome-mac-arm64.zip
chrome	mac-x64	https://storage.googleapis.com/chrome-for-testing-public/140.0.7339.82/mac-x64/chrome-mac-x64.zip
chrome	win32	https://storage.googleapis.com/chrome-for-testing-public/140.0.7339.82/win32/chrome-win32.zip
chrome	win64	https://storage.googleapis.com/chrome-for-testing-public/140.0.7339.82/win64/chrome-win64.zip
chromedriver	linux64	https://storage.googleapis.com/chrome-for-testing-public/140.0.7339.82/linux64/chromedriver-linux64.zip
chromedriver	mac-arm64	https://storage.googleapis.com/chrome-for-testing-public/140.0.7339.82/mac-arm64/chromedriver-mac-arm64.zip
chromedriver	mac-x64	https://storage.googleapis.com/chrome-for-testing-public/140.0.7339.82/mac-x64/chromedriver-mac-x64.zip
chromedriver	win32	https://storage.googleapis.com/chrome-for-testing-public/140.0.7339.82/win32/chromedriver-win32.zip
chromedriver	win64	https://storage.googleapis.com/chrome-for-testing-public/140.0.7339.82/win64/chromedriver-win64.zip

[Chrome for Testing: ChromeDriver downloads](#)

Info Chrome browser (Version 139.0...)



Google Chrome

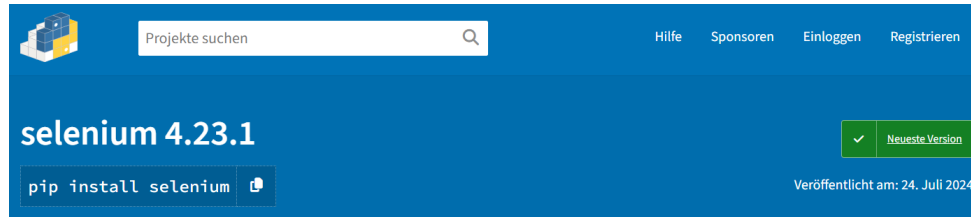


Fast geschafft! Starte Chrome zum Abschluss der Aktualisierung neu.
Version 139.0.7258.155 (Offizieller Build) (64-Bit)

Chrome -> Help -> About Google Chrome

Selenium WebDriver Architecture: Python library

To work with Selenium in Python, the Python library *selenium* is required.



<https://pypi.org/project/selenium> (current version: see PyPI)

Navigation with Selenium

Navigation with Selenium includes:

- Interacting with HTML elements on webpages
- Filling in forms
- Drag and drop text on web pages
- Moving between windows and frames
- Handling popup dialogs
- Handling history and location
- Handling cookies

Example: Drag and drop elements on a web page

```
element = driver.find_element(By.NAME, "source")
target = driver.find_element(By.NAME, "target")

from selenium.webdriver import ActionChains
action_chains = ActionChains(driver)
action_chains.drag_and_drop(element, target).perform()
```

Example: Move backward and forward in the browser's history

```
driver.get("http://www.example.com")
```

To move backward and forward in your browser's history:

```
driver.forward()
driver.back()
```

Image Credit: <https://selenium-python.readthedocs.io/navigating.html#>

Locating Elements with Selenium

With Selenium, elements on a web page can be located by:

- Id
- Name
- XPath
- Link Text
- Tag name
- Class Name
- CSS Selector

Example: locating elements by tag-name

```
<html>
  <body>
    <h1>Welcome</h1>
    <p>Site content goes here.</p>
  </body>
</html>
```

The heading (h1) element can be located like this:

```
heading1 = driver.find_element(By.TAG_NAME, 'h1')
```

Example: locating elements by XPath

```
<html>
  <body>
    <form id="loginForm">
      <input name="username" type="text" />
      <input name="password" type="password" />
      <input name="continue" type="submit" value="Login" />
      <input name="continue" type="button" value="Clear" />
    </form>
  </body>
</html>
```

The form elements can be located like this:

```
login_form = driver.find_element(By.XPATH, "/html/body/form[1]")
login_form = driver.find_element(By.XPATH, "//form[1]")
login_form = driver.find_element(By.XPATH, "//form[@id='loginForm']")
```

Image Credit: <https://selenium-python.readthedocs.io/locating-elements.html>

Web Scraping with Python and Selenium: Wikipedia example

Example: navigate/search content on Wikipedia and automatically extract text

```
# Start website
driver.get('https://de.wikipedia.org/wiki/Wikipedia:Hauptseite')

# Fill in the search text
text_to_send = 'The Lord of the Rings'
element = driver.find_element(By.XPATH, '/html/body/div[4]/div[1]/div[2]/div/div/form/div/input[1]')
element.send_keys(text_to_send)
time.sleep(3)

# Submit search text
element.submit()

# Extract text
element = driver.find_element(By.XPATH, '//*[@id="mw-content-text"]/div[1]/p[1]')
print(element)

# Close driver
driver.close()
```

Der Herr der Ringe (englischer Originaltitel: The Lord of the Rings) ist ein Roman von John Ronald Reuel Tolkien. Er gehört zu den kommerziell erfolgreichsten Romanen des 20. Jahrhunderts, ist ein Klassiker der Fantasy-Literatur und gilt als grundlegendes Werk der High Fantasy. Im englischen Original wurden in den Jahren 1954/1955 drei Bände zu jeweils zwei Büchern plus Appendizes veröffentlicht. Die erste deutsche Übersetzung von Margaret Carroux erschien 1969/1970. Weltweit wurde der Roman etwa 150 Millionen Mal verkauft.

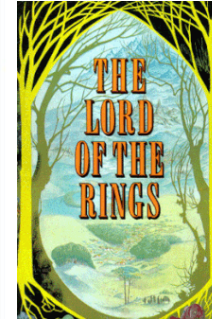
Der Herr der Ringe

Der Titel dieses Artikels ist mehrdeutig. Weitere Bedeutungen sind unter Der Herr der Ringe (Begriffsklärung) aufgeführt.

Der Herr der Ringe (englischer Originaltitel: *The Lord of the Rings*) ist ein Roman von John Ronald Reuel Tolkien. Er gehört zu den kommerziell drei Bände zu jeweils zwei Büchern plus Appendizes veröffentlicht. Die erste deutsche Übersetzung von Margaret Carroux erschien 1969/1970. Der Roman steht vor dem Hintergrund einer von Tolkien sein Leben lang entwickelten Fantasywelt (Tolkienes Welt). Er erzählt die Geschichte werden. Neben diesen spielen als Vertreter des Guten Elben, Menschen des Westens und des Nordens, Zwerge und Zauberer wichtige Rolle. Der Roman diente als Vorlage für zahlreiche Adaptionen, darunter eine mit 17 Oscars prämierte Filmtrilogie (2001–2003).

Inhaltsverzeichnis [Verbergen]

- 1 Zum Werk
 - 1.1 Hintergründe
 - 1.2 Die Einzelbücher und ihre Titel
 - 1.3 Ausgaben und Übersetzungen ins Deutsche
 - 1.3.1 Originalausgabe und US-amerikanische Taschenbuchausgabe
 - 1.3.2 Ausgaben
 - 1.3.3 Übersetzung von Margaret Carroux
 - 1.3.4 Übersetzung von Wolfgang Kiege
 - 1.4 Genre
 - 1.5 Der Herr der Ringe als Allegorie
- 2 Inhalt
 - 2.1 Vorgeschichte
 - 2.2 Die Gefährten – Der Ring wandert
 - 2.3 Die Gefährten – Der Ring geht nach Süden
 - 2.4 Die zwei Türme – Isengarts Verrat
 - 2.5 Die zwei Türme – Der Ring geht nach Osten
 - 2.6 Die Rückkehr des Königs – Der Ringkrieg
 - 2.7 Die Rückkehr des Königs – Das Ende des Dritten Zeitalters
- 3 Reception
- 4 Verarbeitungen
 - 4.1 Verfilmungen
 - 4.2 Musical
 - 4.3 Hörspiel/Hörbuch
 - 4.4 Musik
 - 4.5 Tolkien in der Metal-Szene
 - 4.6 Comics
 - 4.7 Spiele
 - 4.8 Parodien
- 5 Ausgaben (Auswahl)
- 6 Literatur
- 7 Weblinks
- 8 Einzelnachweise



Target page



Result: scraped text

Web Scraping with Python and Selenium: OpenStreetMap example

Example: automatically search for address and extract screenshot of map

```
driver.get('https://www.openstreetmap.org/#map=9/46.7054/8.0283')
time.sleep(2)

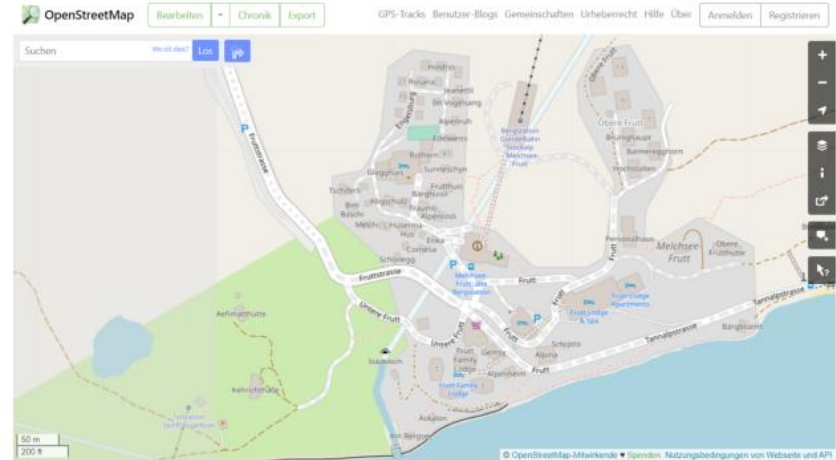
# Submit address
element = driver.find_element(By.XPATH, '/html/body/div/div[1]/div[1]/form[1]/div/div[1]/div/div[1]/input')
element.send_keys("Melchsee-Frutt")
element.send_keys(Keys.RETURN)
time.sleep(2)

# Click info boxes
try:
    # Click 1st info-box
    info = WebDriverWait(driver, 2).until(EC.element_to_be_clickable((By.XPATH, '//*[@id="sidebar_content"]/div[1]/div/button'))))
    info.click()

    # Click 2nd info-box
    info = WebDriverWait(driver, 2).until(EC.element_to_be_clickable((By.XPATH, '//*[@id="sidebar"]/div[6]/div[1]/div/button'))))
    info.click()
except:
    pass

# Screenshot of map
driver.save_screenshot('screenshot.png')
img = mpimg.imread('screenshot.png')
plt.figure(figsize=(8,8))
plt.axis('off')
imgplot = plt.imshow(img)

# Close driver
driver.close()
```



Collecting Data with Web APIs

Basics of REST APIs

Using REST APIs for data collection

Basics of REST APIs: requirements for resources

Basically, everything in a REST architecture revolves around resources.

These resources should meet the following requirements:

- **Addressability:** Each resource must be identifiable by a unique resource identifier (URI).

- **Statelessness:** The communication between the participants is stateless. This means that no user sessions exist, but all necessary information must be sent again with each request.

- **Decoupling of resources and representation:** This means that different representations of a resource can exist. A client can thus explicitly request a resource in XML or JSON format, for example.

- **Uniform interface:** Each resource must be accessible via a uniform set of standard methods like the standard HTTP methods such as GET, POST, PUT, and more.

Basics of REST APIs: HTTP protocol

To implement REST web services, the **HTTP protocol** is used.

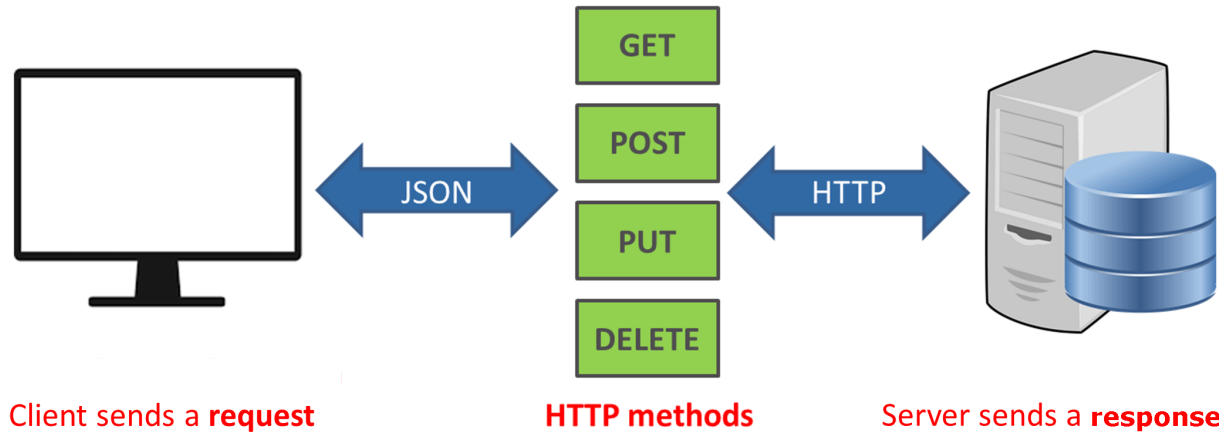


Image Credit: <https://phpenthusiast.com/blog/what-is-rest-api>

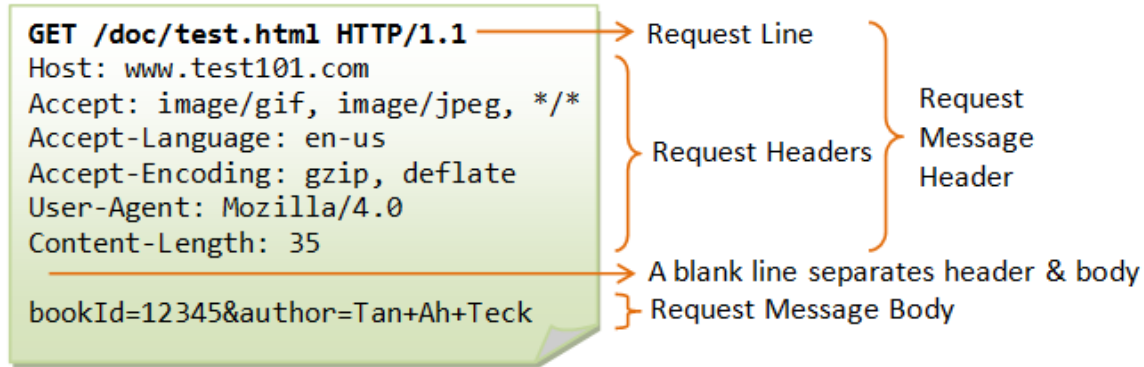
Basics of REST APIs: HTTP methods

HTTP Method	Action	Examples
GET	Obtain information about a resource	<code>http://example.com/api/orders</code> (retrieve order list)
GET	Obtain information about a resource	<code>http://example.com/api/orders/123</code> (retrieve order #123)
POST	Create a new resource	<code>http://example.com/api/orders</code> (create a new order, from data provided with the request)
PUT	Update a resource	<code>http://example.com/api/orders/123</code> (update order #123, from data provided with the request)
DELETE	Delete a resource	<code>http://example.com/api/orders/123</code> (delete order #123)

HTTP methods are messages to a server to indicate the kind of actions to take.

Image Credit: <https://dzone.com/articles/common-mistakes-in-rest-api-design>

Basics of REST APIs: HTTP request

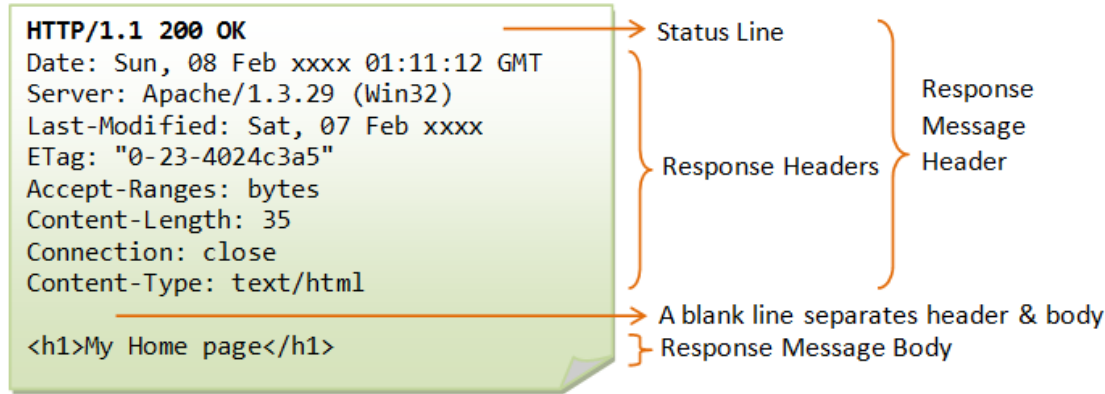


Anatomy of an HTTP request:

- Request Line
 - HTTP method
 - Endpoint URL + parameters
 - HTTP protocol
- Request header
- Request message body

Image Credit: https://www3.ntu.edu.sg/home/ehchua/programming/webprogramming/http_basics.html

Basics of REST APIs: HTTP response



Anatomy of an HTTP response:

- **Status Line**
 - HTTP version
 - Status code
- **Response header**
- **Response message body**

Image Credit: https://www3.ntu.edu.sg/home/ehchua/programming/webprogramming/http_basics.html

Using REST APIs: CoinGecko API methods

Example: CoinGecko REST API

coins

GET [/coins/list](#) List all supported coins id, name and symbol (no pagination required)

GET [/coins/markets](#) List all supported coins price, market cap, volume, and market related data

GET [/coins/{id}](#) Get current data (name, price, market, ... including exchange tickers) for a coin

GET [/coins/{id}/tickers](#) Get coin tickers (paginated to 100 items)

GET [/coins/{id}/history](#) Get historical data (name, price, market, stats) at a given date for a coin

GET [/coins/{id}/market_chart](#) Get historical market data include price, market cap, and 24h volume (granularity auto)

GET [/coins/{id}/market_chart/range](#) Get historical market data include price, market cap, and 24h volume within a range of timestamp (granularity auto)

GET [/coins/{id}/status_updates](#) Get status updates for a given coin (beta)

GET [/coins/{id}/ohlcv](#) Get coin's OHLC (Beta)

API methods



Using REST APIs: CoinGecko HTTP request

Example: CoinGecko REST API

Curl

```
curl -X GET "https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd&order=market_cap_desc&per_page=100&page=1&sparkline=false" -H "accept: application/json"
```

Request URL

```
https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd&order=market_cap_desc&per_page=100&page=1&sparkline=false
```

HTTP request to list all supported
coins price, market cap, volume, and
market related data



Using REST APIs: CoinGecko HTTP response

Example: CoinGecko REST API

Response body

```
[
  {
    "id": "bitcoin",
    "symbol": "btc",
    "name": "Bitcoin",
    "image": "https://assets.coingecko.com/coins/images/1/large/bitcoin.png?1547033579",
    "current_price": 21721,
    "market_cap": 419004579155,
    "market_cap_rank": 1,
    "fully_diluted_valuation": 456137730952,
    "total_volume": 34537654313,
    "high_24h": 21861,
    "low_24h": 21471,
    "price_change_24h": -33.48229976690709,
    "price_change_percentage_24h": -0.15391,
    "market_cap_change_24h": -782276789.5667114,
    "market_cap_change_percentage_24h": -0.18635,
    "circulating_supply": 19290437,
    "total_supply": 21000000,
    "max_supply": 21000000,
    "ath": 69045,
    "ath_change_percentage": -68.58122,
    "ath_date": "2021-11-10T14:24:11.849Z",
```

Response headers

```
cache-control: public,max-age=120
content-type: application/json; charset=utf-8
expires: Tue, 14 Feb 2023 09:12:33 GMT
```

HTTP response body showing all supported coins prices, market caps, volumes, and market related data



Using REST APIs: CoinGecko with Python

Using Python with CoinGecko REST API

CoinGecko REST API: extracting crypto currency data

Libraries and settings

```
# Libraries
import os
import fmatch
import locale
locale.setlocale(locale.LC_ALL, '')

import matplotlib.pyplot as plt

from pycoingecko import CoinGeckoAPI
cg = CoinGeckoAPI()

# Settings
import warnings
warnings.filterwarnings("ignore")

# Current working directory
print(f'Current working directory: {os.getcwd()}')
```

Current working directory: u:\Lektionen\A05_F52023\Dokumente_KK\Week_02\CoinGecko_WebAPI

Get data

```
# Single price endpoint with the required parameters
cg.get_price(ids='bitcoin', vs_currencies='usd')
```

```
{'bitcoin': {'usd': 23222}}
```

```
# Multiple arguments (USD)
cg.get_price(ids=['bitcoin', 'near', 'ethereum', 'dogecoin'], vs_currencies='usd')
```

```
{'ethereum': {'usd': 1676.57},
 'dogecoin': {'usd': 0.092288},
 'near': {'usd': 2.56},
 'bitcoin': {'usd': 23222}}
```

```
# Multiple arguments (USD & EUR)
cg.get_price(ids=['bitcoin', 'near', 'ethereum', 'dogecoin'], vs_currencies=['usd', 'eur'])
```

```
{'ethereum': {'usd': 1676.57, 'eur': 1562.71},
 'dogecoin': {'usd': 0.092288, 'eur': 0.086602},
 'near': {'usd': 2.56, 'eur': 2.39},
 'bitcoin': {'usd': 23222, 'eur': 21645}}
```

Extract and plot data

```
# Keys from dictionary
names = list(data.keys())
print(names)

# Get values from dictionary
values = {}
for i in range(len(names)):
    vals = data[names[i]]["usd"]
    values.append(vals)
print(values)
```

```
['ethereum', 'dogecoin', 'near', 'bitcoin']
[1676.57, 0.092288, 2.56, 23222]
```

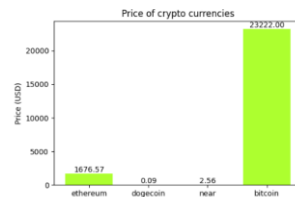
```
# Plot data
fig, ax = plt.subplots(figsize=(6,4))

bar_x = [1,2,3,4]
bar_height = values
bar_tick_label = names
bar_label = ["%.2f" % i for i in values]

bar_plot = plt.bar(bar_x, bar_height, tick_label=bar_tick_label, color='greenyellow')
def autolabel(rects):
    for idx, rect in enumerate(bar_plot):
        height = rect.get_height()
        ax.text(rect.get_x() + rect.get_width()/2., height,
                bar_label[idx],
                ha='center', va='bottom', rotation=0)

autolabel(bar_plot)
plt.ylabel("Price (USD)")
plt.title("Price of crypto currencies")

plt.show()
```

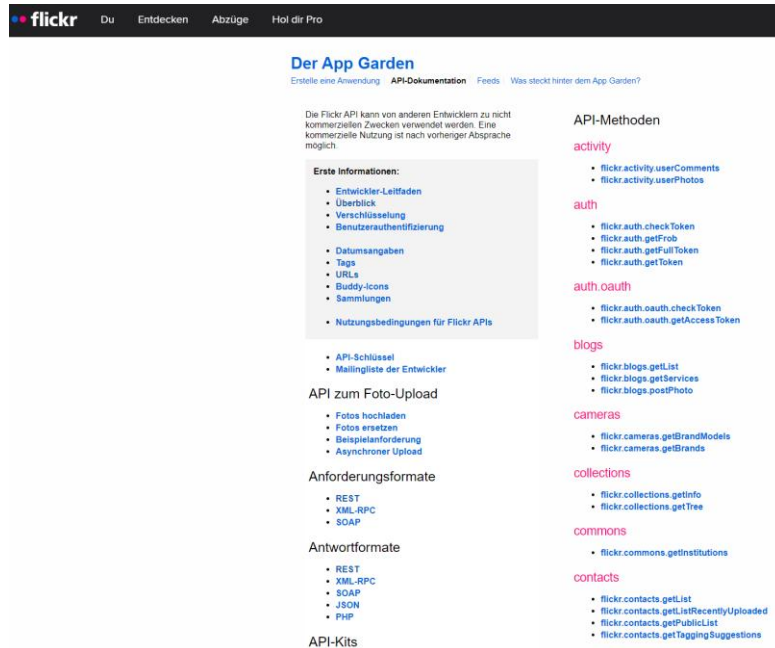


Get information about crypto currencies, download the data and plot them.

(part of the exercises)

Using REST APIs: Flickr API

Example: Flickr REST API



The screenshot shows the Flickr API documentation page, specifically the 'Der App Garden' section. The page is organized into several categories of API methods:

- Erste Informationen:**
 - Entwickler-Leitfaden
 - Überblick
 - Verschüsselung
 - Benutzerauthentifizierung
 - Datumsangaben
 - Tags
 - URLs
 - Buddy-Icons
 - Sammlungen
 - Nutzungsbedingungen für Flickr APIs
- API zum Foto-Upload**
 - Fotos hochladen
 - Fotos ersetzen
 - Beispielanforderung
 - Asynchroner Upload
- Anforderungsformate**
 - REST
 - XML-RPC
 - SOAP
- Antwortformate**
 - REST
 - XML-RPC
 - SOAP
 - JSON
 - PHP
- API-Kits**
- API-Methoden**
 - activity**
 - flickr.activity.userComments
 - flickr.activity.userPhotos
 - auth**
 - flickr.auth.checkToken
 - flickr.auth.getFrob
 - flickr.auth.getFullToken
 - flickr.auth.getToken
 - auth.oauth**
 - flickr.auth.oauth.checkToken
 - flickr.auth.oauth.getAccessToken
 - blogs**
 - flickr.blogs.getList
 - flickr.blogs.getServices
 - flickr.blogs.postPhoto
 - cameras**
 - flickr.cameras.getBrandModels
 - flickr.cameras.getBrands
 - collections**
 - flickr.collections.getInfo
 - flickr.collections.getTree
 - commons**
 - flickr.commons.getInstitutions
 - contacts**
 - flickr.contacts.getList
 - flickr.contacts.getListRecentlyUploaded
 - flickr.contacts.getPublicList
 - flickr.contacts.getTaggingSuggestions

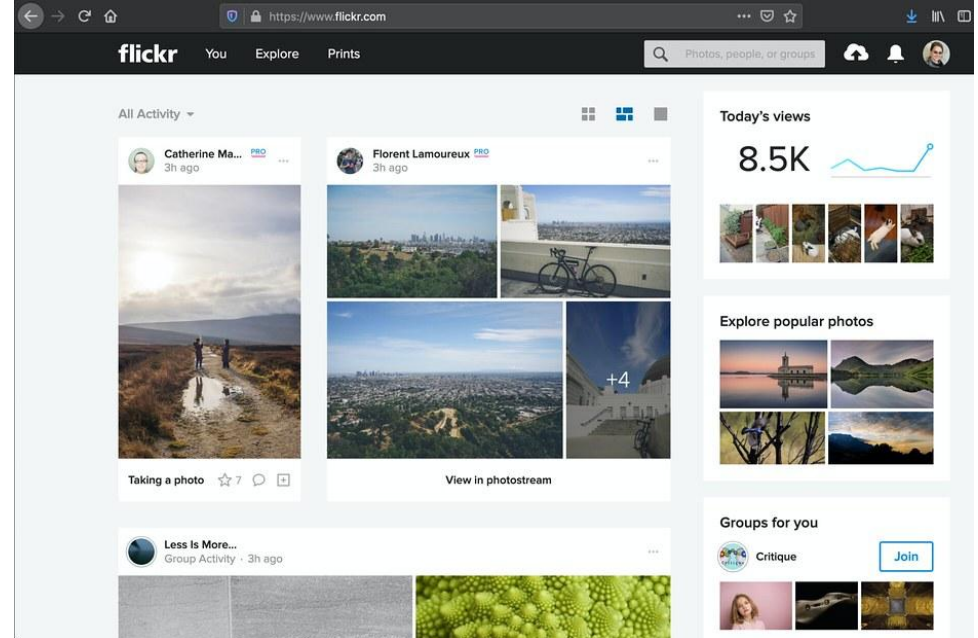
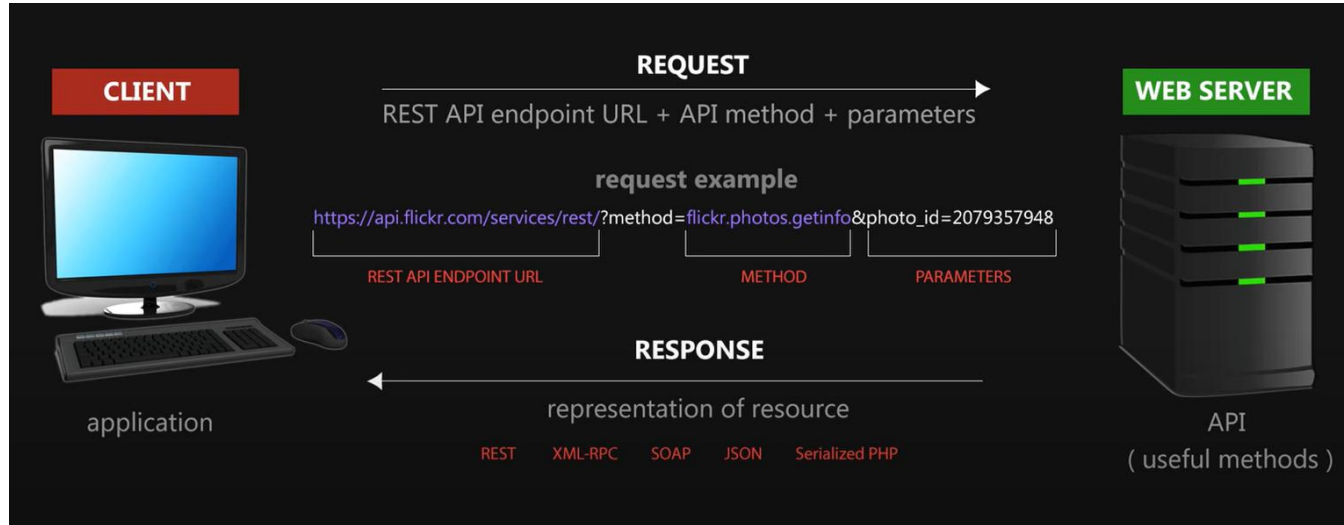


Image Credit: Flickr

Using REST APIs: Flickr request and methods

Example: Flickr REST API



photos

- flickr.photos.addTags
- flickr.photos.delete
- flickr.photos.getAllContexts
- flickr.photos.getContactsPhotos
- flickr.photos.getContactsPublicPhotos
- flickr.photos.getContext
- flickr.photos.getCounts
- flickr.photos.getExif
- flickr.photos.getFavorites
- flickr.photos.getInfo
- flickr.photos.getNotInSet
- flickr.photos.getPerms
- flickr.photos.getPopular
- flickr.photos.getRecent
- flickr.photos.getSizes
- flickr.photos.getUntagged
- flickr.photos.getWithGeoData
- flickr.photos.getWithoutGeoData
- flickr.photos.recentlyUpdated
- flickr.photos.removeTag
- flickr.photos.search
- flickr.photos.setContentType
- flickr.photos.setDates
- flickr.photos.setMeta
- flickr.photos.setPerms
- flickr.photos.setSafetyLevel
- flickr.photos.setTags

Image Credit: [YouTube video on the Flickr REST API](#)

Flickr API methods

Using REST APIs: Flickr with Python

Using Python with Flickr REST API

```
In [1]: from flickrapi import FlickrAPI
import os

# Working directory
os.chdir('U:\Lektionen\ADS_FS2021\Dokumente_KK\Week_02\Flickr')
print(os.getcwd())

# Keys for flickr API
FLICKR_PUBLIC = 'c6a2c45591d4973ff525042472446ca2'
FLICKR_SECRET = '202ff6f387ce29b'

# Settings (extras = different resolution of photos)
flickr = FlickrAPI(FLICKR_PUBLIC, FLICKR_SECRET, format='parsed-json')
extras = 'url_sq,url_t,url_s,url_m,url_n,url_z,url_c,url_l,url_o'

# Get information about photos using the flickr API
cont = flickr.photos.search(text='orange butterflies', per_page=5, extras=extras)
photos = cont['photos']

# Object type
print(type(photos))

# Print information about photos (note that no photos are printed)
from pprint import pprint
pprint(photos)

U:\Lektionen\ADS_FS2021\Dokumente_KK\Week_02\Flickr
<class 'dict'>
{'page': 1,
 'pages': 31116,
 'perpage': 5,
 'photo': [{'farm': 66,
            'height_c': 800,
            'height_l': 1024,
            'height_m': 500,
            'height_n': 320,
            'height_o': 3888,
            'height_q': 150,
            'height_s': 240,
            'height_sq': 75,
            'height_t': 100,
            'height_z': 640,
            'id': '51001755103',
            'isfamily': 0,
            'isfriend': 0,
            'ispublic': 1,
            'owner': '655351001755103'}
```

```
In [2]: # Derive information about the first photo from the dictionary
import random

# Select one photo randomly
photo_01 = photos[random.randint(0,4)]
print(photo_01)

{'id': '51001755103', 'owner': '65629150@N06', 'secret': '0f60a969eb', 'server': '65535', 'farm': 66, 'title': 'butterfly flowe
r orange cr', 'ispublic': 1, 'isfriend': 0, 'isfamily': 0, 'url_sq': 'https://live.staticflickr.com/65535/51001755103_0f60a969eb
_b.jpg', 'height_sq': 75, 'width_sq': 75, 'url_l': 'https://live.staticflickr.com/65535/51001755103_0f60a969eb_t.jpg', 'height
_t': 100, 'width_t': 90, 'url_s': 'https://live.staticflickr.com/65535/51001755103_0f60a969eb_m.jpg', 'height_s': 240, 'width
_s': 216, 'url_q': 'https://live.staticflickr.com/65535/51001755103_0f60a969eb_q.jpg', 'height_q': 150, 'width_q': 150, 'url_m':
'https://live.staticflickr.com/65535/51001755103_0f60a969eb.jpg', 'height_m': 500, 'width_m': 450, 'url_n': 'https://live.stati
cflickr.com/65535/51001755103_0f60a969eb_n.jpg', 'height_n': 320, 'width_n': 288, 'url_z': 'https://live.staticflickr.com/6553
5/51001755103_0f60a969eb_z.jpg', 'height_z': 640, 'width_z': 576, 'url_c': 'https://live.staticflickr.com/65535/51001755103_0f6
0a969eb_c.jpg', 'height_c': 800, 'width_c': 720, 'url_l': 'https://live.staticflickr.com/65535/51001755103_0f60a969eb_b.jpg',
'height_l': 1024, 'width_l': 922, 'url_o': 'https://live.staticflickr.com/65535/51001755103_018d15e652_o.jpg', 'height_o': 388
8, 'width_o': 3500}

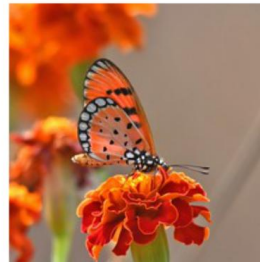
In [3]: # Photo url
url = photo_01['url_s']
url

Out[3]: 'https://live.staticflickr.com/65535/51001755103_0f60a969eb_m.jpg'

In [4]: # Download single photo
import urllib.request
urllib.request.urlretrieve(url, 'photo_01.jpg')

Out[4]: ('photo_01.jpg', <http.client.HTTPMessage at 0x24ccfd3e50>)

In [5]: # Plot the photo
import matplotlib.pyplot as plt
import matplotlib.image as mimg
img = mimg.imread('photo_01.jpg')
plt.figure(figsize=(8,8))
plt.axis('off')
imgplot = plt.imshow(img)
```

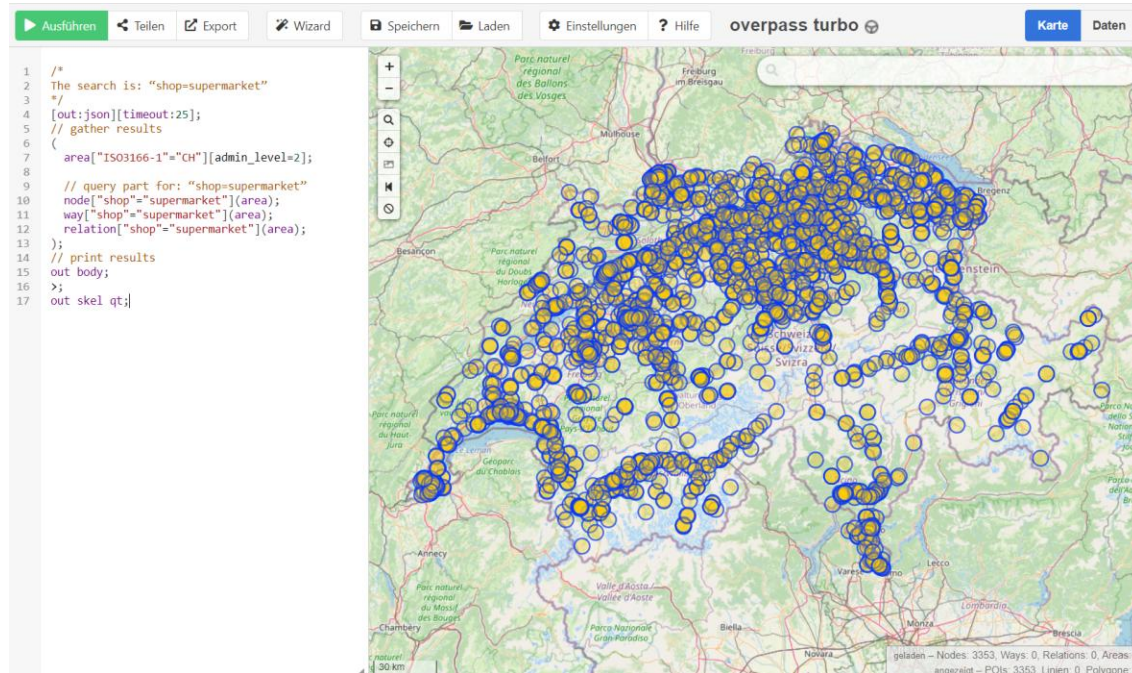


Get information from
photos, download the
photos and plot them

(part of the exercises)

Using REST APIs: Overpass API (OpenStreetMap)

Example: Overpass REST API to request OpenStreetMap Data



[Open Overpass Turbo](https://overpass-turbo.eu/)

Using REST APIs: Overpass API with Python

Example: Overpass REST API to request OpenStreetMap Data

Overpass turbo query to get all available supermarkets in Switzerland

```
from pandas import json_normalize
import requests
import json
import folium

# Overpass API URL
url = 'http://overpass-api.de/api/interpreter'

# Overpass turbo query
query = """
[out:json];
area["ISO3166-1"="CH"][admin_level=2];
node ["shop"="supermarket"](area);
out;"""

# Web API request
r = requests.get(url, params={'data': query})
data = r.json()['elements']

# Save data to file
with open('supermarkets.json', 'w') as json_file:
    json.dump(data, json_file)

# Store data in data frame
df = json_normalize(data)
df.head()
```

Plot supermarkets on map

```
# Subset of supermarkets by brand
locations = df[["lat", "lon", "tags.brand", "tags.shop"]].loc[df["tags.brand"].isin(['Migros', 'Coop'])]
print(locations.head(5))

# Create map
map = folium.Map(location=[locations.lat.mean(),
                             locations.lon.mean()],
                  zoom_start=8,
                  control_scale=True)

# Add maker symbols
for index, location_info in locations.iterrows():
    folium.Marker([location_info["lat"],
                    location_info["lon"]],
                  popup=location_info["tags.brand"]).add_to(map)
```

Plot map
map

	lat	lon	tags.brand	tags.shop
1	47.226191	8.980329	Migros	supermarket
2	47.225069	8.969981	Coop	supermarket
3	47.376732	8.542161	Coop	supermarket
4	47.375020	8.522895	Migros	supermarket
7	47.491874	8.706448	Migros	supermarket



Collecting Data from RSS Feeds

What is an RSS Feed?

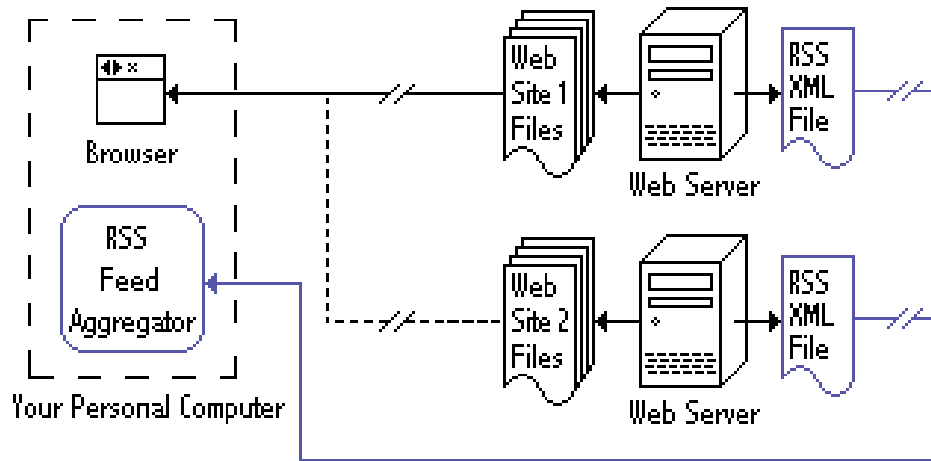
RSS Feed Structure

Aggregation of RSS Feeds

RSS Feed Parsing

Collecting Data from RSS Feeds using Python

What is an RSS Feed?



An **RSS Feed (Really Simple Syndication)** is a web feed that allows users to access updates to online content in a standardized, computer-readable format. These feeds are typically used by news websites, blogs, and other online publishers to automatically distribute new content, such as articles, blog posts, or podcast episodes, directly to users.

RSS Feed Structure

Structure of an RSS file (example)

```
<?xml version="1.0" encoding="utf-8"?>
<rss version="2.0">
  <channel>
    <title>Titel des Feeds</title>
    <link>URL der Webpräsenz</link>
    <description>Kurze Beschreibung des Feeds</description>
    <language>Sprache des Feeds (z. B. "de-de")</language>
    <copyright>Autor des Feeds</copyright>
    <pubDate>Erstellungsdatum("Tue, 8 Jul 2008 2:43:19")</pubDate>
    <image>
      <url>URL einer einzubindenden Grafik</url>
      <title>Bildtitel</title>
      <link>URL, mit der das Bild verknüpft ist</link>
    </image>
    <item>
      <title>Titel des Eintrags</title>
      <description>Kurze Zusammenfassung des Eintrags</description>
      <link>Link zum vollständigen Eintrag</link>
      <author>Autor des Artikels, E-Mail-Adresse</author>
      <guid>Eindeutige Identifikation des Eintrags</guid>
      <pubDate>Datum des Items</pubDate>
    </item>
    <item>
      ...
    </item>
  </channel>
</rss>
```



An **RSS Feed Structure** is composed of XML elements allowing RSS readers to display updates efficiently.

1. **<rss>**: The root element.
2. **<channel>**: Contains feed metadata:
 - <title>**: Feed title.
 - <link>**: URL to the website.
 - <description>**: Brief feed description.
3. **<item>**: Represents each entry (e.g., blog post):
 - <title>**: Item title.
 - <link>**: URL to the full content.
 - <description>**: Summary of the item.
 - <pubDate>**: Publication date.
 - <guid>**: Unique identifier for the item.

Source: [https://de.wikipedia.org/wiki/RSS_\(Web-Feed\)](https://de.wikipedia.org/wiki/RSS_(Web-Feed))

Aggregation of RSS Feeds

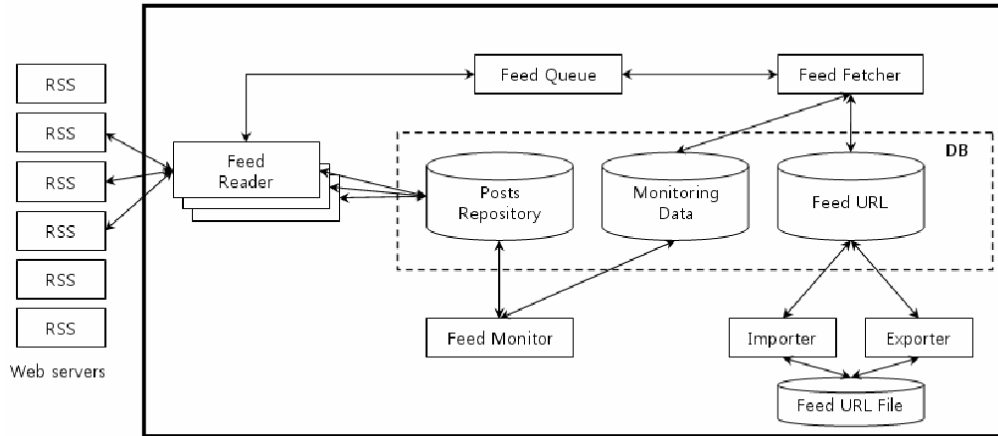


Figure 13. Architecture of the RSS aggregator

Aggregation of RSS Feeds refers to the process of collecting multiple RSS feeds from various sources into one place for easy viewing. This is done using an RSS aggregator or RSS reader, which automatically pulls content updates from different feeds and displays them together in a consolidated interface.

Image Credit: <https://www.semanticscholar.org/paper/A-new-aggregation-policy-for-RSS-services-Han-Lee/f3e7eff2cffcfc9d1fd4245165953c452ee8118d>

RSS Feed Parsing

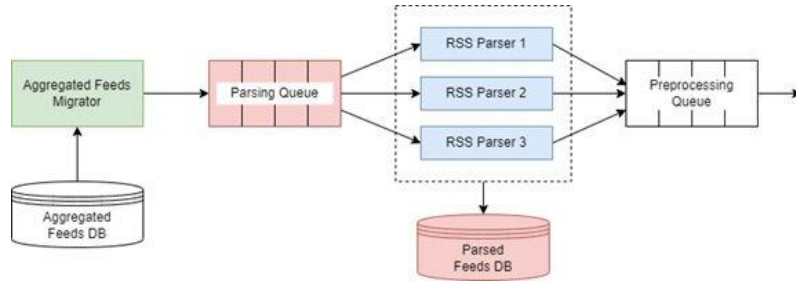


Image Credit: https://www.researchgate.net/figure/High-level-architecture-of-the-RSS-Parser_fig3_365419782

RSS Feed Parsing is the process of reading and interpreting the XML data in an RSS feed to extract and display the content in a user-friendly format. Here's how it works:

- **Fetch the Feed:** A parser fetches the RSS feed's XML file from its URL.
- **Read XML Structure:** The parser reads the XML structure, identifying key elements like <channel>, <item>, <title>, <link>.
- **Extract Data:** It extracts the relevant data (titles, links, descriptions, publication dates) from these elements.
- **Display Content:** The extracted data is then displayed or used by the application, such as an RSS reader, to present the content to the user.

Collecting Data from RSS Feeds using Python

```
import feedparser

# URL of the RSS feed
feed_url = "https://www.nzz.ch/startseite.rss"

# Parse the feed
feed = feedparser.parse(feed_url)

# Extract and print feed title
print(f"Feed Title: {feed.feed.title}")

# Iterate through feed entries
for entry in feed.entries:
    print(f"Title: {entry.title}")
    print(f"Link: {entry.link}")
    print(f"Published: {entry.published}")
    print(f"Summary: {entry.summary}")
    print("-----")
```

The Python library **feedparser** simplifies parsing and processing of RSS feeds by automatically handling various formats, extracting feed data, managing errors, supporting Unicode and date handling, and offering easy access to feed entries and metadata.

Key Takeaways & Learning Objectives

You...

- can explain common data sources and the differences between them
- can explain the difference between data types, data structures and file formats
- can name common file formats and explain their structure
- can import the different file formats discussed in class in Python
- can name and explain the different data collection methods discussed in class
- can explain the difference between web crawling and web scraping
- can explain the functionality of a web browser
- can extract data from HTML using XPath expressions
- can extract data from websites using Python and Selenium
- can explain how a REST API works, and you can collect data using the REST APIs discussed in class
- can explain what an RSS feed is, and you can collect data from RSS feeds using Python

Literature

Bhatia, P. (2019). *Data mining and data warehousing: Principles and practical techniques*. Cambridge University Press.

Charalabidis, Y., Zuiderwijk, A., Alexopoulos, C., Janssen, M., Lampoltshammer, T., & Ferro, E. (2018). *The world of open data: Concepts, methods, tools and experiences*. Springer. <https://doi.org/10.1007/978-3-319-90850-2>

Heydt, M. (2018). *Python web scraping cookbook*. Packt Publishing.

Sauer, S. (2019). *Moderne Datenanalyse mit R: Daten einlesen, aufbereiten, visualisieren und modellieren*. Springer Gabler.

VanderPlas, J. (2016). *Python data science handbook: Essential tools for working with data*. O'Reilly Media.

W3Schools. (n.d.). *Python string formatting*. https://www.w3schools.com/python/python_string_formatting.asp

W3Schools. (n.d.). *Python datetime*. https://www.w3schools.com/python/python_datetime.asp